

PROJETO VIGIAR · ENTREGA DE COMPONENTE PYTHON

ETL — Extração, Transformação e Carga dos Dados do VigiAr

Pipeline de tratamento de dados de saúde respiratória e clima para o estado de São Paulo

GRUPO

DataSquad

INSTITUIÇÃO

FIAP — Turma 1TSCPF

COMPONENTE ENTREGUE

Etapa de ETL (Extração, Transformação e Carga)

DATA

Agosto de 2026

INTEGRANTES

Halen

Igor Soares

João Marcos Borba

João Ricardo Travaglin

Vitor Franco Benvenuto

Sumário

1. Visão geral do código
2. Funcionamento do código, passo a passo
3. Justificativa da abordagem adotada
4. Evidências de execução (prints)
5. Apêndice — código-fonte completo

1. Visão geral do código

Esse código é um **ETL** — um programa que pega dados brutos, limpa e organiza esses dados, e junta tudo numa única tabela pronta para uso. No caso do VigiAr, ele baixa quatro fontes de dados de saúde e clima de São Paulo, trata cada uma separadamente, e no final entrega um CSV único: um retrato de cada município paulista, semana a semana, com casos de síndrome respiratória grave, leitos disponíveis, população e temperatura. Esse CSV é o que alimenta o modelo preditivo do projeto, responsável por tentar prever surtos respiratórios com antecedência.

O arquivo entregue é um notebook do Google Colab (`etl_vigiar_colab_v3.py`), organizado em blocos que rodam em sequência: download dos dados, configuração de funções auxiliares, inspeção inicial, tratamento de cada uma das quatro fontes, classificação geográfica dos municípios, união de tudo numa tabela só e, por fim, o cálculo das colunas finais e o salvamento do resultado.

As quatro fontes de dados tratadas são:

- **SRAG (SIVEP-Gripe)** — notificações de síndrome respiratória grave, a base principal;
- **Leitos hospitalares** — capacidade de atendimento de cada município;
- **População (IBGE)** — usada para calcular taxas por habitante;
- **Temperatura (INMET)** — usada porque o clima influencia a sazonalidade de doenças respiratórias.

2. Funcionamento do código, passo a passo

2.1 — Baixar os dados brutos

O código entra num espaço de armazenamento na nuvem (Oracle Cloud) e baixa todos os arquivos disponíveis. Antes de baixar cada arquivo, ele checa se esse arquivo já existe na pasta local — isso é importante porque o Google Colab às vezes reinicia sozinho por inatividade, e sem essa checagem seria preciso baixar tudo de novo do zero toda vez.

2.2 — Preparação (funções de apoio)

Antes de mexer nos dados de verdade, o código define um conjunto de ferramentas que vai reaproveitar depois: uma função para padronizar nomes de município (sem acento, em maiúsculo), uma para formatar datas no padrão brasileiro, e uma leitura de CSV “à prova de falha” — ela tenta ler o arquivo de duas formas diferentes de codificação de texto e, se o separador de colunas não for vírgula (alguns arquivos usam ponto e vírgula), detecta isso sozinha em vez de quebrar.

Também é aqui que ficam os dicionários de tradução: os dados de saúde vêm com respostas em código numérico — por exemplo, o campo de vacinação vem como 1, 2 ou 9. O código traduz isso para "Sim", "Não" e "Ignorado", usando o dicionário oficial do SIVEP-Gripe.

2.3 — Etapa 0: Inspeção

Antes de tratar qualquer coisa, o código dá uma olhada na estrutura de cada arquivo (quais colunas existem, como estão os dados). Isso ajuda a confirmar que tudo está no formato esperado antes de seguir em frente, evitando erros de interpretação mais adiante.

2.4 — Etapa 1: Temperatura (INMET)

Lê os arquivos das estações meteorológicas de São Paulo, extrai a latitude e longitude de cada uma e calcula a temperatura média por semana. Essas coordenadas servem depois tanto para preencher a temperatura de municípios sem estação própria quanto para classificar a região de cada município.

2.5 — Etapa 2: SRAG (a base principal)

Essa é a etapa mais pesada do código. Primeiro, filtra só os casos de São Paulo. Depois, traduz todos os campos codificados (se o paciente foi internado em UTI, se tomou vacina, se teve febre, se o teste deu positivo, entre outros) e calcula indicadores derivados — por exemplo, “esse caso é de covid?”, “esse paciente é idoso?”, “quantos dias demorou entre o início dos sintomas e a internação?”. No final, agrupa tudo por município e semana, transformando casos individuais em números agregados: total de casos, taxa de óbito, taxa de vacinação, e assim por diante.

2.6 — Etapa 3: Leitos hospitalares

Filtra os dados de leitos só de São Paulo e soma, por município, quantos leitos SUS existem (total, UTI adulto, UTI pediátrica, UTI neonatal), calculando também a proporção de leitos que são do SUS e quantos hospitais distintos cada município tem.

2.7 — Etapa 4: População (IBGE)

Lê um arquivo JSON com a população de cada município — no formato aninhado que vem direto da API do IBGE — e extrai só o que interessa: nome do município e população em 2025.

2.8 — Classificação da região (Norte / Sul / Leste / Oeste / Centro)

Em vez de usar uma tabela fixa dizendo qual município é de qual região, o código calcula isso sozinho: pega a coordenada de todos os municípios, acha o “centro geográfico” do estado (a mediana das coordenadas) e classifica

cada município comparando sua posição com esse centro.

| 2.9 — Montagem da grade completa e junção de tudo

Aqui o código garante que **todo município tenha uma linha para toda semana do período**, mesmo que não tenha havido nenhum caso naquela semana — nesse caso, o valor vira zero em vez de a linha simplesmente não existir. Isso é importante porque “não tem essa linha” e “teve zero casos” são informações bem diferentes para um modelo preditivo. Depois disso, o código junta as quatro fontes numa tabela só.

| 2.10 — Geração do dataset final

Aqui acontece a geolocalização de fato (busca a coordenada de cada município), a temperatura que faltava é preenchida usando a estação mais próxima, e são calculadas as últimas colunas derivadas: casos a cada 10 mil habitantes, leitos a cada 1.000 habitantes, casos da semana anterior, média móvel de 3 semanas, variação de temperatura e variação percentual de casos. No final, o código salva tudo num CSV.

| 2.11 — Download do resultado

Por fim, o código baixa o CSV final para o computador de quem está rodando o notebook.

3. Justificativa da abordagem adotada

Por que Python com pandas?

É a ferramenta padrão do mercado para esse tipo de trabalho — juntar dados de formatos bem diferentes (CSV com vírgula, CSV com ponto e vírgula, JSON aninhado) e transformar tudo numa tabela só. Fazer isso na mão, ou numa planilha, seria inviável com o volume de dados do projeto.

Por que granularidade semanal, e não mensal?

O objetivo do VigiAr é avisar um gestor de saúde sobre um possível surto **com antecedência**. Se os dados fossem agregados por mês, um sinal de alerta ficaria diluído e a resposta seria mais lenta. A semana é o equilíbrio entre ter dado suficiente para calcular uma média confiável e não perder tempo de reação.

Por que traduzir os códigos numéricos antes de contar?

Deixa o processo auditável — qualquer pessoa lendo o código entende na hora o que cada valor significa, sem precisar consultar o manual do SIVEP-Gripe toda vez. Isso reduz erro de interpretação e facilita explicar os resultados para quem não é técnico, que é justamente o público do projeto (gestores de saúde do CROSS/SP).

Por que preencher toda semana com zero (grade completa)?

Evita que o modelo preditivo confunda “não tivemos dado dessa semana” com “não houve caso”. São duas informações diferentes, e misturar isso prejudicaria a qualidade das previsões.

Por que classificar a região de forma automática, em vez de uma lista fixa?

O código se adapta sozinho ao conjunto real de municípios presentes nos dados, em vez de depender de uma lista que alguém precisaria manter manualmente. Essa escolha, inclusive, corrigiu um problema de uma versão anterior do projeto, em que a maioria dos municípios estava caindo incorretamente na região “Centro”.

Quais os benefícios gerais dessa abordagem?

O código é dividido em funções pequenas e independentes — uma para cada fonte de dado — o que facilita testar e corrigir cada parte sem afetar as outras. A leitura de arquivo é resistente a falhas (tenta formatos diferentes antes de desistir), e o download só baixa o que ainda não existe, economizando tempo se a sessão do Colab cair no meio do processo.

Como isso atende à necessidade do projeto?

O VigiAr precisa de uma base de dados limpa, semanal e por município, cruzando saúde, capacidade hospitalar, clima e população — exatamente o que esse ETL entrega. É essa tabela final que alimenta o modelo de machine learning (Random Forest) responsável por tentar prever se vai haver um surto respiratório com pelo menos um mês de antecedência.

4. Evidências de execução

A seguir estão os prints da execução completa do notebook, na ordem em que as células foram rodadas — do download dos dados brutos até a geração e o download do arquivo final.

Início do notebook e função de download

O cabeçalho do notebook explica a versão v3 do ETL. Logo abaixo está a função que baixa os arquivos brutos do armazenamento em nuvem (Oracle Cloud) — ela pula os arquivos que já foram baixados antes, o que evita perder tempo baixando tudo de novo se a sessão do Colab reiniciar no meio do processo.

ETL - VigiAr (v3 — granularidade semanal, região, decodificação de campos)

Pipeline completo: baixa os dados do bucket Oracle Cloud, trata cada fonte (SRAG, Leitos, Temperatura, População), une tudo num único CSV — recorte São Paulo, agregado por **semana** (não mais por mês).

Novidades desta versão em relação à anterior:

- Datas no formato brasileiro (ex: 29/08/2025).
- Campos categóricos do SRAG (ex: EVOLUCAO) decodificados para o significado real, usando o dicionário de dados oficial do SIVEP-Gripe — em vez do código numérico cru.
- Agregação por **semana** em vez de mês.
- Nova coluna **regiao** (Norte/Sul/Leste/Oeste/Centro), com base na posição geográfica do município dentro do estado.
- Mantida a imputação de temperatura por proximidade geográfica para municípios sem estação do INMET.

Como usar: rode as células de cima pra baixo. A célula de download pula arquivos já existentes, então é seguro rodar de novo se a sessão reiniciar no meio do processo.

Download dos dados brutos

1] Os

```
1 # (Opcional, mas recomendado) Monta o Google Drive.
2 # Sem isso, os arquivos baixados somem toda vez que a sessão do Colab reinicia
3 # por inatividade e você precisa baixar tudo de novo.
4
5 # from google.colab import drive
6 # drive.mount('/content/drive')
7
```

2] 20min

```
1 import os
2 import requests
3 from urllib.parse import quote
4
5 BASE_URL = (
6     "https://objectstorage.sa-saopaulo-1.oraclecloud.com/p/"
7     "3RgzWgPKLSx2idIorDnZng9Szc1eF-EaBwY301ePGCOuqIwT_Y26pXJHhmkOw00/"
8     "n/grksbwp7ro2x/b/teste_vigiar/o/"
9 )
10 PASTA_DADOS = "/content/downloads" # AJUSTAR para "/content/drive/MyDrive/..." se usar o Drive
11
12
13 def baixar_todos_os_arquivos():
14     resp = requests.get(BASE_URL)
15     resp.raise_for_status()
16     objetos = [obj["name"] for obj in resp.json()["objects"]]
17     print(f"{len(objetos)} objetos encontrados no bucket. Baixando...")
18
19     for i, nome in enumerate(objetos, start=1):
20         if nome.endswith("/"):
21             continue
22         caminho_local = os.path.join(PASTA_DADOS, nome)
23         os.makedirs(os.path.dirname(caminho_local), exist_ok=True)
24         if os.path.exists(caminho_local):
25             continue
26         with requests.get(BASE_URL + quote(nome), stream=True) as r:
27             r.raise_for_status()
28             with open(caminho_local, "wb") as f:
29                 for chunk in r.iter_content(chunk_size=8192):
30                     f.write(chunk)
31             if i % 25 == 0 or i == len(objetos):
32                 print(f" {i}/{len(objetos)} processados")
33
34     print("Download concluído. Arquivos em:", PASTA_DADOS)
35
36
37 baixar_todos_os_arquivos()
38
```


Resultado do download

O log confirma que os 600 objetos do bucket foram encontrados e baixados com sucesso para a pasta /content/downloads.

```
... 600 objetos encontrados no bucket. Baixando...
  25/600 processados
  50/600 processados
  75/600 processados
 100/600 processados
 125/600 processados
 150/600 processados
 175/600 processados
 200/600 processados
 225/600 processados
 250/600 processados
 275/600 processados
 300/600 processados
 325/600 processados
 350/600 processados
 375/600 processados
 400/600 processados
 425/600 processados
 450/600 processados
 475/600 processados
 500/600 processados
 525/600 processados
 550/600 processados
 575/600 processados
 600/600 processados
Download concluído. Arquivos em: /content/downloads
```

Bloco de configuração – parte 1

Aqui ficam as funções auxiliares reaproveitadas no resto do código: padronizar nome de município (maiúsculo, sem acento), formatar datas no padrão brasileiro, e uma leitura de CSV “à prova de falha”, que tenta duas codificações de texto e detecta sozinha se o separador de colunas não é vírgula.



Bloco de configuração — parte 2

Os dicionários de tradução dos campos do SRAG: cada código numérico (1, 2, 9...) vira o seu significado real (“Sim”, “Não”, “Ignorado” etc.), seguindo o dicionário oficial do SIVEP-Gripe. Logo abaixo, a função que aplica essa tradução em qualquer coluna.

```

59
60 # — Dicionários de decodificação (dicionário de dados oficial SIVEP-Gripe) —
61 # Aplicados linha a linha, ANTES da agregação por semana/município — o CSV
62 # final continua com contagens/taxas numéricas, só que calculadas em cima do
63 # significado real do campo, não do código cru. Isso deixa o processo mais
64 # auditável (fica claro no código o que cada valor representa) e evita erro
65 # de interpretação de código.
66
67 MAPA_SIM_NAO = {1: "Sim", 2: "Não", 9: "Ignorado"}
68 MAPA_UTI = {1: "Sim", 2: "Não", 9: "Ignorado"}
69 MAPA_VACINA = {1: "Sim", 2: "Não", 9: "Ignorado"}
70 MAPA_TP_IDADE = {1: "Dia", 2: "Mês", 3: "Ano"}
71 MAPA_EVOLUCAO = {1: "Cura", 2: "Óbito", 3: "Óbito por outras causas", 9: "Ignorado"}
72 MAPA_CLASSI_FIN = {
73     1: "SRAG por influenza",
74     2: "SRAG por outro vírus respiratório",
75     3: "SRAG por outro agente etiológico",
76     4: "SRAG não especificado",
77     5: "SRAG por covid-19",
78 }
79 MAPA_SUPORT_VEN = {1: "Sim, invasivo", 2: "Sim, não invasivo", 3: "Não", 9: "Ignorado"}
80 MAPA_PCR_RESUL = {1: "Detectável", 2: "Não detectável", 3: "Inconclusivo",
81                   4: "Não realizado", 5: "Aguardando resultado", 9: "Ignorado"}
82 MAPA_RES_AN = {1: "Positivo", 2: "Negativo", 3: "Inconclusivo",
83               4: "Não realizado", 5: "Aguardando resultado", 9: "Ignorado"}
84 MAPA_CS_GESTANT = {1: "1º trimestre", 2: "2º trimestre", 3: "3º trimestre",
85                   4: "Idade gestacional ignorada", 5: "Não", 6: "Não se aplica", 9: "Ignorado"}
86 MAPA_CS_ZONA = {1: "Urbana", 2: "Rural", 3: "Periurbana", 9: "Ignorado"}
87
88
89 def decodificar_categoria(serie, mapa):
90     """Converte uma coluna de códigos numéricos (ex: 1, 2, 9) para o
91     significado real (ex: 'Sim', 'Não', 'Ignorado'), usando o dicionário de
92     dados oficial. Valores ausentes ou códigos fora do mapa viram
93     'Não informado', em vez de ficar em branco."""
94     numerico = pd.to_numeric(serie, errors="coerce")
95     return numerico.map(mapa).fillna("Não informado")
96
97
98 def serie_segura(df, coluna):
99     """Devolve a coluna se ela existir no DataFrame; senão devolve uma coluna
100     vazia (tudo NaN) e avisa — assim uma variável faltando não derruba o
101     pipeline inteiro, só fica em branco no resultado final."""
102     if coluna not in df.columns:
103         print(f"Aviso: coluna '{coluna}' não encontrada no CSV — as variáveis derivadas dela ficarão vazias.")
104         return pd.Series([None] * len(df), index=df.index)
105     return df[coluna]
106
107
108 1 SAIDA = "/content/dataset_vigiar_sp.csv" # ajuste para /content/drive/... se estiver usando o Drive
109 2

```

Etapa 0 — Inspeção (parte 1)

Antes de tratar qualquer dado, o código “espia” a estrutura real de cada arquivo — quais colunas existem, como estão organizadas — para não trabalhar às cegas.

Etapa 0 — Inspeção

Confere a estrutura real de cada arquivo antes de tratar qualquer coisa.

```

1 def encontrar_linha_cabecalho(caminho, encoding="latin1"):
2     """Procura a linha cujo PRIMEIRO campo (antes do ';') é exatamente 'Data'.
3     Não basta checar se a linha começa com 'Data', porque arquivos do INMET têm
4     uma linha de metadado 'DATA DE FUNDACAO;...' que também começaria com
5     'Data' e seria confundida com o cabeçalho real da tabela."""
6     with open(caminho, encoding=encoding) as f:
7         for i, linha in enumerate(f):
8             primeiro_campo = linha.split(";")[0].strip().upper()
9             if primeiro_campo == "DATA":
10                 return i
11     return 8
12
13
14 def extrair_coordenadas_estacao(caminho, encoding="latin1"):
15     """Lê a latitude/longitude da estação, presentes no bloco de metadados
16     no topo do arquivo."""
17     lat, lon = None, None
18     with open(caminho, encoding=encoding) as f:
19         for linha in f:
20             primeiro_campo = linha.split(";")[0].strip().upper()
21             if primeiro_campo.startswith("LATITUDE"):
22                 try:
23                     lat = float(linha.split(";")[1].strip().replace(", ", "."))
24                 except (IndexError, ValueError):
25                     pass
26             elif primeiro_campo.startswith("LONGITUDE"):
27                 try:
28                     lon = float(linha.split(";")[1].strip().replace(", ", "."))
29                 except (IndexError, ValueError):
30                     pass
31             if primeiro_campo == "DATA":
32                 break
33     return lat, lon
34
35
36 def encontrar_coluna(colunas, texto_procurado):
37     """Retorna a primeira coluna cujo nome contém o texto procurado (case-insensitive)."""
38     texto_procurado = texto_procurado.upper()
39     for c in colunas:
40         if texto_procurado in str(c).upper():
41             return c
42     return None
43
44
45 def inspecionar_csv(caminho, n_linhas=10, **kwargs):
46     print(f"\n{'='*70}\n{caminho}\n{'='*70}")
47     df = ler_csv_seguro(caminho, nrows=n_linhas, **kwargs)
48     print("Colunas:", list(df.columns))
49     display(df.head())
50
51
52 def inspecionar_json(caminho):
53     print(f"\n{'='*70}\n{caminho}\n{'='*70}")
54     with open(caminho, encoding="utf-8") as f:
55         dados = json.load(f)
56         amostra = dados[:3] if isinstance(dados, list) else dados
57         print(json.dumps(amostra, indent=2, ensure_ascii=False)[:1500])
58

```

Etapa 0 — Inspeção (parte 2)

Função que mostra as primeiras linhas dos arquivos de temperatura do INMET (que têm um formato mais complicado, com metadados no topo), e a chamada que roda a inspeção de todos os arquivos de uma vez.

```

58
59
60 def inspecionar_inmet(caminho, n_linhas=15):
61     print(f"\n{' '*70}\n{caminho} (bruto, primeiras linhas)\n{' '*70}")
62     with open(caminho, encoding="latin1") as f:
63         for i, linha in enumerate(f):
64             print(linha.strip())
65             if i >= n_linhas:
66                 break
67
68
69 def inspecionar_tudo():
70     inspecionar_csv(os.path.join(PASTA_DADOS, "Leitos_2025.csv"))
71     inspecionar_csv(os.path.join(PASTA_DADOS, "srag_2025_completo.csv"))
72     inspecionar_json(os.path.join(PASTA_DADOS, "cnes_estabelecimentos.json"))
73     inspecionar_json(os.path.join(PASTA_DADOS, "populacao_municipios_2025 (1).json"))
74     exemplos_sp = glob.glob(os.path.join(PASTA_DADOS, "TEMPERATURA", "INMET_SE_SP_*.CSV"))
75     if exemplos_sp:
76         inspecionar_inmet(exemplos_sp[0])
77     else:
78         print("Não encontrei CSVs de temperatura de SP em", os.path.join(PASTA_DADOS, "TEMPERATURA"))
79
1 inspecionar_tudo()
2

```

Resultado da inspeção inicial

Saída real da inspeção: as colunas e as primeiras linhas dos arquivos de Leitos, do SRAG e do JSON de estabelecimentos de saúde (CNES), confirmando que os arquivos estão no formato esperado antes de seguir em frente.

1 inspecionar_tudo()
2

/content/downloads/leitos_2025.csv

Colunas: ['comp', 'regiao', 'uf', 'municipio', 'motivo_desabilitacao', 'cnes', 'nome_estabelecimento', 'razao_social', 'tp_gestao', 'co_tipo_unidade', 'ds_tipo_unidade', 'natureza_juridica', 'desc_natureza_juridica', 'no_logradouro', 'nu_endereco', 'no_complemento', 'no_bairro', 'co_ccp', 'nu_telefone', 'no_email', 'lei...

	comp	regiao	uf	municipio	motivo_desabilitacao	cnes	nome_estabelecimento	razao_social	tp_gestao	co_tipo_unidade	ds_tipo_unidade	uti_adulto_exist	uti_adulto_sus	uti_pediatico_exist	uti_pediatico_sus	uti_neonatal_exist	uti_neonatal_sus	uti_queimado_exist	uti_queimado_sus
0	202501	NORDESTE	PE	CABO DE SANTO AGOSTINHO		NaN	27	CASA DE SAUDE SANTA HELENA		CASA DE SAUDE E MATERNIDADE SANTA HELENA LTDA		M	5	...	0	0	0	0	0
1	202501	NORDESTE	PE	CABO DE SANTO AGOSTINHO		NaN	35	HOSPITAL MENDO SAMPAIO		PREFEITURA MUNICIPAL DO CABO DE SANTO AGOSTINHO		M	5	...	0	0	0	0	0
2	202501	NORDESTE	PE	CABO DE SANTO AGOSTINHO		NaN	94	MATERNIDADE PADRE GERALDO LEITE BASTOS		PREFEITURA MUNICIPAL DO CABO DE SANTO AGOSTINHO		M	7	...	0	0	0	0	0
3	202501	NORDESTE	PE	CABO DE SANTO AGOSTINHO		NaN	183	HOSPITAL SAMARITANO		SOCIEDADE HOSPITALAR SAMARITANO LTDA		M	5	...	5	0	0	0	0
4	202501	NORDESTE	PE	CABO DE SANTO AGOSTINHO		NaN	221	HOSPITAL SAO SEBASTIAO		CASA DE SAUDE E MATERNIDADE SAO SEBASTIAO LTDA		M	5	...	10	0	0	0	0

5 rows x 34 columns

/content/downloads/srag_2005_completo.csv

Aviso: srag_2005_completo.csv não usa ',' como separador - detectando automaticamente...
Colunas: ['nu_notific', 'dt_notific', 'spa_not', 'dt_sin_pri', 'spa_pri', 'co_sf_not', 'id_regional', 'co_regional', 'id_municip', 'co_mun_not', 'cs_sexo', 'dt_nasc', 'nu_idade_n', 'tp_idade', 'cod_idade', 'cs_gestant', 'cs_raca', 'cs_etnia', 'cs_escol_n', 'id_pais', 'co_pais', 'sg_uf', 'id_rg_resi', 'co_rg_resi', 'id_mu...

	nu_notific	dt_notific	spa_not	dt_sin_pri	spa_pri	co_sf_not	id_regional	co_regional	id_municip	co_mun_not	...	vs_oms	vs_omsout	vs_ltn	vs_met	vs_metout	vs_dtres	vs_enc	vs_reinf	vs_cdest	reinf
0	31743527606384	2025-03-16T00:00:00.000Z	12	2025-03-13T00:00:00.000Z	11	SP	OVE VII SANTO ANDRE	1332.0	SAO CAETANO DO SUL	354880	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2
1	31755901798720	2025-08-09T00:00:00.000Z	32	2025-07-07T00:00:00.000Z	28	RR		NaN	NaN	BOA VISTA	140010	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2
2	31743372054067	2025-03-30T00:00:00.000Z	14	2025-03-29T00:00:00.000Z	13	PB	III NRS CAMPINA GRANDE	1421.0	CAMPINA GRANDE	250400	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2
3	31740491434244	2025-02-25T00:00:00.000Z	9	2025-02-24T00:00:00.000Z	9	PR	02RS METROPOLITANA	1358.0	CURITIBA	410690	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2
4	31750101341771	2025-06-16T00:00:00.000Z	25	2025-06-13T00:00:00.000Z	24	CE	13 CRES TIANGUA	1523.0	VICOSA DO CEARA	231410	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2

5 rows x 194 columns

/content/downloads/cnes_estabelecimentos.json

{
 {
 "co_cnes": "19",
 "co_unidade": "2682900000019",
 "co_uf": "26",
 "co_bairro": "268290",
 "nu_cnpj_mantenedora": "11294402000162",
 "no_razao_social": "PREFEITURA MUNICIPAL DO CABO DE SANTO AGOSTINHO",
 "no_fantasia": "POLICLINICA DR JAMALI DE MEDEIROS",
 "co_natureza_organizacao": "1",
 "ds_natureza_organizacao": "",
 "...

Etapla 1 — Temperatura (INMET)

Função que lê os arquivos de todas as estações meteorológicas de São Paulo, extrai a coordenada de cada uma e calcula a temperatura média por semana — base para preencher depois a temperatura de municípios sem estação própria.

▼ Etapla 1 — Temperatura (INMET, agregada por semana)

Mesma leitura robusta de antes (detecção automática de cabeçalho, colunas de data/temperatura), mas agora agregada por **semana** em vez de mês. Também extrai a latitude/longitude de cada estação — usadas mais adiante tanto para a imputação de temperatura quanto para a classificação regional dos municípios.

```
[?]
✓ Os
1 def carregar_temperatura_sp():
2     arquivos = glob.glob(os.path.join(PASTA_DADOS, "TEMPERATURA", "INMET_SE_SP_*.CSV"))
3     print(f"{len(arquivos)} estações de SP encontradas")
4
5     tabelas = []
6     falhas = []
7     estacoes_coords = {}
8     for caminho in arquivos:
9         nome_arquivo = os.path.basename(caminho)
10        try:
11            partes = nome_arquivo.replace(".CSV", "").split("_")
12            codigo_estacao = partes[3]
13            cidade_estacao = normalizar_municipio(partes[4].split(" - ")[0])
14
15            lat, lon = extrair_coordenadas_estacao(caminho)
16            estacoes_coords[cidade_estacao] = {"lat": lat, "lon": lon}
17
18            linha_cabecalho = encontrar_linha_cabecalho(caminho)
19
20            df = pd.read_csv(
21                caminho, sep=";", decimal=".", skiprows=linha_cabecalho,
22                encoding="latin1", engine="python", on_bad_lines="skip",
23            )
24            df["codigo_estacao"] = codigo_estacao
25            df["cidade_estacao"] = cidade_estacao
26            tabelas.append(df)
27        except Exception as e:
28            falhas.append((nome_arquivo, str(e)))
29
30    if falhas:
31        print(f"\n{len(falhas)} arquivo(s) não puderam ser lidos e foram pulados:")
32        for nome, erro in falhas:
33            print(f" - {nome}: {erro}")
34
35    temperatura = pd.concat(tabelas, ignore_index=True)
36
37    coluna_data_real = encontrar_coluna(temperatura.columns, "DATA")
38    coluna_temp = encontrar_coluna(temperatura.columns, "TEMPERATURA DO AR")
39
40    if coluna_data_real is None or coluna_temp is None:
41        print("Não encontrei automaticamente as colunas de data/temperatura.")
42        print("Colunas disponíveis:", list(temperatura.columns))
43        raise KeyError("Ajuste coluna_data_real / coluna_temp manualmente com um dos nomes acima.")
44
45    temperatura["data"] = pd.to_datetime(temperatura[coluna_data_real], errors="coerce")
46    temperatura["semana"] = temperatura["data"].dt.to_period("W")
47
48    resumo_semanal = (
49        temperatura
50        .groupby(["cidade_estacao", "semana"])[coluna_temp]
51        .mean()
52        .reset_index()
53        .rename(columns={coluna_temp: "temp_media", "cidade_estacao": "municipio"})
54    )
55    print("Temperatura: OK")
56    return resumo_semanal, estacoes_coords
57
```

Etapla 2 — SRAG, início

Início do tratamento da base principal: filtra só os casos de São Paulo e começa a traduzir os campos mais importantes — diagnóstico final, evolução do caso, UTI, vacina e faixa de idade.

▼ Etapla 2 — SRAG (base principal, filtrada para SP, agregada por semana)

Os campos categóricos (`CLASSI_FIN`, `EVOLUCAO`, `UTI`, `VACINA`, comorbidades, `TP_IDADE`) são decodificados para o significado real antes de qualquer contagem — usando os dicionários definidos na Configuração. Códigos ausentes ou fora do padrão viram "Não informado". As contagens e taxas finais (`casos_influenza`, `taxa_obito`, etc.) são calculadas em cima desses valores decodificados, não dos códigos numéricos crus.

Também extraímos `regional_saude` (campo `ID_REGIONA`) — a Regional de Saúde oficial que o SUS usa para organizar referência hospitalar entre municípios, diferente da classificação geográfica simples (`regiao` : Norte/Sul/Leste/Oeste/Centro) calculada mais adiante.

```
[E]
✓ Os
1 def carregar_srag():
2     print("Carregando SRAG...")
3     df = ler_csv_seguro(os.path.join(PASTA_DADOS, "srag_2025_completo.csv"))
4
5     coluna_uf = "SG_UF_NOT"
6     coluna_data = "DT_NOTIFIC"
7     coluna_municipio_resi = "ID_MN_RESI" # município de RESIDÊNCIA do paciente (chave geográfica)
8     coluna_municipio_notif = "ID_MUNICIP" # município de NOTIFICAÇÃO (mantido só de referência)
9     coluna_regional = "ID_REGIONA" # Regional de Saúde de notificação (agrupamento oficial do SUS)
10
11     for nome_var, coluna in [
12         ("coluna_uf", coluna_uf), ("coluna_data", coluna_data),
13         ("coluna_municipio_resi", coluna_municipio_resi),
14         ("coluna_municipio_notif", coluna_municipio_notif),
15     ]:
16         if coluna not in df.columns:
17             print(f"\nA coluna '{coluna}' (usada como {nome_var}) não existe no arquivo.")
18             print("Colunas disponíveis:", list(df.columns))
19             raise KeyError(f"Ajuste {nome_var} em carregar_srag() com um dos nomes acima.")
20
21     df = df[df[coluna_uf] == UF_ALVO]
22
23     df["data_notificacao"] = pd.to_datetime(df[coluna_data], errors="coerce")
24     df["semana"] = df["data_notificacao"].dt.to_period("W")
25
26     # Checagem: ID_MN_RESI precisa vir como NOME de município (texto), igual
27     # ID_MUNICIP, pra normalizar_municipio() funcionar. Se vier como código
28     # numérico do IBGE, o aviso abaixo aparece — nesse caso normalizar_municipio
29     # não resolve sozinho, precisaria de uma tabela código -> nome antes.
30     amostra = df[coluna_municipio_resi].dropna().astype(str).head(5).tolist()
31     print(f"Amostra de valores em {coluna_municipio_resi}: {amostra}")
32     if amostra and all(v.replace(".", "").isdigit() for v in amostra):
33         print(f"ATENÇÃO: '{coluna_municipio_resi}' parece conter CÓDIGOS numéricos do IBGE, não "
34               f"nomes de município — normalizar_municipio() não vai funcionar direito nesse caso.")
35
36     df["municipio"] = df[coluna_municipio_resi].apply(normalizar_municipio) # chave = residência
37     df["municipio_notificacao"] = df[coluna_municipio_notif].apply(normalizar_municipio) # mantido pra comparação
38
39     if coluna_regional in df.columns:
40         df["regional_saude"] = df[coluna_regional].fillna("Não informado")
41     else:
42         print(f"Aviso: coluna '{coluna_regional}' não encontrada — regional_saude ficará como 'Não informado'.")
43         df["regional_saude"] = "Não informado"
44
45     # — Decodificação dos campos categóricos (dicionário oficial SIVEP-Gripe) —
46     df["classi_fin_decodificado"] = decodificar_categoria(df["CLASSI_FIN"], MAPA_CLASSI_FIN)
47     df["evolucao_decodificada"] = decodificar_categoria(df["EVOLUCAO"], MAPA_EVOLUCAO)
48     df["uti_decodificado"] = decodificar_categoria(df["UTI"], MAPA_UTI)
49     df["vacina_decodificado"] = decodificar_categoria(df["VACINA"], MAPA_VACINA)
50     df["tp_idade_decodificado"] = decodificar_categoria(df["TP_IDADE"], MAPA_TP_IDADE)
51
```


Etapla 2 — SRAG, comorbidades e sintomas

Continuação da tradução dos campos: comorbidades (diabetes, asma, obesidade etc.), sintomas (febre, tosse, falta de ar), testes realizados e seus resultados — e o cálculo dos indicadores derivados de cada caso (por exemplo, “eh_covid”, “foi_vacinado”).

```

52     columnas_comorbidade = [
53         "CARDIOPATI", "DIABETES", "ASMA", "OBESIDADE",
54         "IMUNODEPRE", "RENAL", "PNEUMOPATI", "NEUROLOGIC",
55     ]
56     for coluna in columnas_comorbidade:
57         df[f"{coluna.lower()}_decodificado"] = decodificar_categoria(df[coluna], MAPA_SIM_NAO)
58
59     # — Decodificação dos campos novos (gravidade, testagem, risco, sintomas) —
60     df["surto_decodificado"] = decodificar_categoria(serie_segura(df, "SURTO_SEG"), MAPA_SIM_NAO)
61     df["saturacao_decodificada"] = decodificar_categoria(serie_segura(df, "SATURACAO"), MAPA_SIM_NAO)
62     df["dispneia_decodificada"] = decodificar_categoria(serie_segura(df, "DISPNEIA"), MAPA_SIM_NAO)
63     df["desc_resp_decodificado"] = decodificar_categoria(serie_segura(df, "DESC_RESP"), MAPA_SIM_NAO)
64     df["suport_ven_decodificado"] = decodificar_categoria(serie_segura(df, "SUPOORT_VEN"), MAPA_SUPORT_VEN)
65     df["pcr_resul_decodificado"] = decodificar_categoria(serie_segura(df, "PCR_RESUL"), MAPA_PCR_RESUL)
66     df["res_an_decodificado"] = decodificar_categoria(serie_segura(df, "RES_AN"), MAPA_RES_AN)
67     df["amostra_decodificada"] = decodificar_categoria(serie_segura(df, "AMOSTRA"), MAPA_SIM_NAO)
68     df["fator_risc_decodificado"] = decodificar_categoria(serie_segura(df, "FATOR_RISC"), MAPA_SIM_NAO)
69     df["cs_gestant_decodificado"] = decodificar_categoria(serie_segura(df, "CS_GESTANT"), MAPA_CS_GESTANT)
70     df["tabag_decodificado"] = decodificar_categoria(serie_segura(df, "TABAG"), MAPA_SIM_NAO)
71     df["cs_zona_decodificada"] = decodificar_categoria(serie_segura(df, "CS_ZONA"), MAPA_CS_ZONA)
72     df["vacina_cov_decodificada"] = decodificar_categoria(serie_segura(df, "VACINA_COV"), MAPA_SIM_NAO)
73     df["febre_decodificada"] = decodificar_categoria(serie_segura(df, "FEBRE"), MAPA_SIM_NAO)
74     df["tosse_decodificada"] = decodificar_categoria(serie_segura(df, "TOSSE"), MAPA_SIM_NAO)
75     df["an_sars2_decodificado"] = decodificar_categoria(serie_segura(df, "AN_SARS2"), MAPA_SIM_NAO)
76     df["pcr_sars2_decodificado"] = decodificar_categoria(serie_segura(df, "PCR_SARS2"), MAPA_SIM_NAO)
77     df["an_vsr_decodificado"] = decodificar_categoria(serie_segura(df, "AN_VSR"), MAPA_SIM_NAO)
78     df["pcr_vsr_decodificado"] = decodificar_categoria(serie_segura(df, "PCR_VSR"), MAPA_SIM_NAO)
79
80     # — Indicadores derivados, calculados a partir dos valores decodificados —
81     df["eh_influenza"] = df["classi_fin_decodificado"] == "SRAG por influenza"
82     df["eh_covid"] = df["classi_fin_decodificado"] == "SRAG por covid-19"
83     df["eh_obito"] = df["evolucao_decodificada"].isin(["Óbito", "Óbito por outras causas"])
84     df["foi_uti"] = df["uti_decodificado"] == "Sim"
85     df["foi_vacinado"] = df["vacina_decodificado"] == "Sim"
86
87     tem_comorbidade = pd.Series(False, index=df.index)
88     for coluna in columnas_comorbidade:
89         tem_comorbidade = tem_comorbidade | (df[f"{coluna.lower()}_decodificado"] == "Sim")
90     df["tem_comorbidade"] = tem_comorbidade
91
92     # Idade: NU_IDADE_N só representa "anos" quando TP_IDADE decodificado == "Ano"
93     idade_em_anos = pd.to_numeric(df["NU_IDADE_N"], errors="coerce").where(
94         df["tp_idade_decodificado"] == "Ano"
95     )
96     df["eh_idoso"] = idade_em_anos >= 60
97
98     # — Indicadores derivados novos —
99     df["eh_surto"] = df["surto_decodificado"] == "Sim"
100     df["eh_saturacao_baixa"] = df["saturacao_decodificada"] == "Sim"
101     df["eh_dispneia"] = df["dispneia_decodificada"] == "Sim"
102     df["eh_desc_resp"] = df["desc_resp_decodificado"] == "Sim"
103     df["eh_vent_invasiva"] = df["suport_ven_decodificado"] == "Sim, invasivo"
104     df["eh_pcr_positivo"] = df["pcr_resul_decodificado"] == "Detectável"
105     df["eh_antigenico_positivo"] = df["res_an_decodificado"] == "Positivo"
106     # SARS2/VSR: "Sim" se QUALQUER um dos dois testes (antígeno OU PCR) deu positivo
107     df["eh_sars2"] = (df["an_sars2_decodificado"] == "Sim") | (df["pcr_sars2_decodificado"] == "Sim")
108     df["eh_vsr"] = (df["an_vsr_decodificado"] == "Sim") | (df["pcr_vsr_decodificado"] == "Sim")
109     df["foi_testado"] = df["amostra_decodificada"] == "Sim"
110     df["tem_fator_risco"] = df["fator_risc_decodificado"] == "Sim"
111     df["eh_gestante"] = df["cs_gestant_decodificado"].isin(
112         ["1º trimestre", "2º trimestre", "3º trimestre", "Idade gestacional ignorada"]
113     )
114     df["eh_tabagista"] = df["tabag_decodificado"] == "Sim"
115     df["eh_zona_rural"] = df["cs_zona_decodificada"] == "Rural"
116     df["foi_vacinado_covid"] = df["vacina_cov_decodificada"] == "Sim"
117     df["teve_febre"] = df["febre_decodificada"] == "Sim"
118     df["teve_tosse"] = df["tosse_decodificada"] == "Sim"
119

```

Etapas 2 — Atrasos e agrupamento

Cálculo de quantos dias se passaram entre etapas do atendimento (do início dos sintomas até a notificação, a coleta do exame, a internação) e o agrupamento de todos os casos por município e semana, somando e tirando médias.

```

119
120 # Município de internação, pra medir fluxo de pacientes referenciados pra
121 # fora do próprio município de residência (índice_fluxo_referencia_hospitalar)
122 df["municipio_internacao"] = serie_segura(df, "ID_MUN_INTE").apply(
123     lambda v: normalizar_municipio(v) if pd.notna(v) else v
124 )
125 df["foi_referenciado"] = (
126     df["municipio_internacao"].notna()
127     & (df["municipio_internacao"] != df["municipio"])
128 )
129
130 # Datas auxiliares, usadas nos cálculos de atraso/tempo abaixo
131 df["dt_sin_pri"] = pd.to_datetime(serie_segura(df, "DT_SIN_PRI"), errors="coerce")
132 df["dt_coleta"] = pd.to_datetime(serie_segura(df, "DT_COLETA"), errors="coerce")
133 df["dt_interna"] = pd.to_datetime(serie_segura(df, "DT_INTERNA"), errors="coerce")
134 df["dt_digita"] = pd.to_datetime(serie_segura(df, "DT_DIGITA"), errors="coerce")
135 df["dt_evoluca"] = pd.to_datetime(serie_segura(df, "DT_EVOLUCA"), errors="coerce")
136 df["dt_entuti"] = pd.to_datetime(serie_segura(df, "DT_ENTUTUI"), errors="coerce")
137 df["dt_saiduti"] = pd.to_datetime(serie_segura(df, "DT_SAIDUTI"), errors="coerce")
138
139 df["dias_ate_notificacao"] = (df["data_notificacao"] - df["dt_sin_pri"]).dt.days
140 df["dias_ate_coleta"] = (df["dt_coleta"] - df["dt_sin_pri"]).dt.days
141 df["dias_ate_internacao"] = (df["dt_interna"] - df["dt_sin_pri"]).dt.days
142 df["dias_uti"] = (df["dt_saiduti"] - df["dt_entuti"]).dt.days
143 df["dias_ate_digitacao"] = (df["dt_digita"] - df["data_notificacao"]).dt.days
144 df["dias_ate_desfecho"] = (df["dt_evoluca"] - df["dt_interna"]).dt.days
145
146 agregados = df.groupby(["municipio", "semana"]).agg(
147     regional_saude=("regional_saude", "first"),
148     casos_srag=("municipio", "size"),
149     casos_influenza=("eh_influenza", "sum"),
150     casos_covid=("eh_covid", "sum"),
151     obitos=("eh_obito", "sum"),
152     casos_uti=("foi_uti", "sum"),
153     casos_comorbidade=("tem_comorbidade", "sum"),
154     casos_vacinados=("foi_vacinado", "sum"),
155     casos_idosos=("eh_idoso", "sum"),
156     casos_surto=("eh_surto", "sum"),
157     casos_saturacao_baixa=("eh_saturacao_baixa", "sum"),
158     casos_dispneia=("eh_dispneia", "sum"),
159     casos_desc_resp=("eh_desc_resp", "sum"),
160     casos_vent_invasiva=("eh_vent_invasiva", "sum"),
161     tempo_medio_uti_dias=("dias_uti", "mean"),
162     casos_pcr_positivo=("eh_pcr_positivo", "sum"),
163     casos_antigenico_positivo=("eh_antigenico_positivo", "sum"),
164     casos_sars2=("eh_sars2", "sum"),
165     casos_vsr=("eh_vsr", "sum"),
166     casos_testados=("foi_testado", "sum"),
167     atraso_notificacao_dias=("dias_ate_notificacao", "mean"),
168     atraso_coleta_dias=("dias_ate_coleta", "mean"),
169     atraso_internacao_dias=("dias_ate_internacao", "mean"),
170     casos_fator_risco=("tem_fator_risco", "sum"),
171     casos_gestantes=("eh_gestante", "sum"),
172     casos_tabagistas=("eh_tabagista", "sum"),
173     casos_zona_rural=("eh_zona_rural", "sum"),
174     casos_vacinados_covid=("foi_vacinado_covid", "sum"),
175     casos_febre=("teve_febre", "sum"),
176     casos_tosse=("teve_tosse", "sum"),
177     atraso_digitacao_dias=("dias_ate_digitacao", "mean"),
178     tempo_ate_desfecho_dias=("dias_ate_desfecho", "mean"),
179     casos_referenciados=("foi_referenciado", "sum"),
180 ).reset_index()
181

```

Etapla 2 — Taxas finais

Cálculo das taxas finais (percentual de óbito, de ocupação de UTI, de vacinação, de positividade nos testes etc.) em cima dos números já agrupados. A mensagem “SRAG: OK” confirma que essa etapa, a mais longa do código, terminou sem erro.

```

181
182     agregados["taxa_obito"] = agregados["obitos"] / agregados["casos_srag"]
183     agregados["taxa_uti"] = agregados["casos_uti"] / agregados["casos_srag"]
184     agregados["taxa_comorbidade"] = agregados["casos_com_comorbidade"] / agregados["casos_srag"]
185     agregados["taxa_vacinados"] = agregados["casos_vacinados"] / agregados["casos_srag"]
186     agregados["taxa_idosos"] = agregados["casos_idosos"] / agregados["casos_srag"]
187
188     agregados["taxa_cadeia_surto"] = agregados["casos_surto"] / agregados["casos_srag"]
189     agregados["taxa_saturacao_baixa"] = agregados["casos_saturacao_baixa"] / agregados["casos_srag"]
190     agregados["taxa_dispneia"] = agregados["casos_dispneia"] / agregados["casos_srag"]
191     agregados["taxa_desc_respiratorio"] = agregados["casos_desc_resp"] / agregados["casos_srag"]
192     agregados["taxa_ventilacao_invasiva"] = agregados["casos_vent_invasiva"] / agregados["casos_srag"]
193     agregados["taxa_positividade_pcr"] = (
194         agregados["casos_pcr_positivo"] / agregados["casos_testados"].replace(0, pd.NA)
195     )
196     agregados["taxa_positividade_antigenico"] = (
197         agregados["casos_antigenico_positivo"] / agregados["casos_testados"].replace(0, pd.NA)
198     )
199     agregados["taxa_sars2"] = agregados["casos_sars2"] / agregados["casos_srag"]
200     agregados["taxa_vsr"] = agregados["casos_vsr"] / agregados["casos_srag"]
201     agregados["taxa_testagem"] = agregados["casos_testados"] / agregados["casos_srag"]
202     agregados["taxa_fator_risco"] = agregados["casos_fator_risco"] / agregados["casos_srag"]
203     agregados["taxa_gestantes"] = agregados["casos_gestantes"] / agregados["casos_srag"]
204     agregados["taxa_tabagismo"] = agregados["casos_tabagistas"] / agregados["casos_srag"]
205     agregados["taxa_zona_rural"] = agregados["casos_zona_rural"] / agregados["casos_srag"]
206     agregados["taxa_vacinacao_covid"] = agregados["casos_vacinados_covid"] / agregados["casos_srag"]
207     agregados["taxa_febre"] = agregados["casos_febre"] / agregados["casos_srag"]
208     agregados["taxa_tosse"] = agregados["casos_tosse"] / agregados["casos_srag"]
209     agregados["indice_fluxo_referencia_hospitalar"] = agregados["casos_referenciados"] / agregados["casos_srag"]
210
211     print("SRAG: OK")
212     return agregados
213

```

Etapla 3 — Leitos hospitalares

Função que filtra os leitos de São Paulo e soma, por município, o total de leitos SUS e a divisão por tipo de UTI (adulto, pediátrica, neonatal), além da proporção de leitos que são do SUS.

▼ Etapla 3 — Leitos (filtrada para SP)

Sem mudança na granularidade temporal (leitos é um dado por município, não por período). Mantém leitos por especialidade de UTI, proporção SUS e contagem de hospitais distintos.

[9]
✓ Os

```
1 def carregar_leitos():
2     print("Carregando Leitos...")
3     df = ler_csv_seguro(os.path.join(PASTA_DADOS, "Leitos_2025.csv"))
4
5     coluna_uf = "UF"
6     coluna_municipio = "MUNICIPIO"
7     coluna_leitos = "LEITOS_SUS"
8     coluna_leitos_existentes = "LEITOS_EXISTENTES"
9     coluna_cnes = "CNES"
10
11     for nome_var, coluna in [
12         ("coluna_uf", coluna_uf), ("coluna_municipio", coluna_municipio), ("coluna_leitos", coluna_leitos),
13     ]:
14         if coluna not in df.columns:
15             print(f"\nA coluna '{coluna}' (usada como {nome_var}) não existe no arquivo.")
16             print("Colunas disponíveis:", list(df.columns))
17             raise KeyError(f"Ajuste {nome_var} em carregar_leitos() com um dos nomes acima.")
18
19     df = df[df[coluna_uf] == UF_ALVO]
20     df["municipio"] = df[coluna_municipio].apply(normalizar_municipio)
21
22     df[coluna_leitos] = pd.to_numeric(df[coluna_leitos], errors="coerce")
23
24     colunas_uti_especialidade = {
25         "UTI_TOTAL_SUS": "leitos_uti_sus",
26         "UTI_ADULTO_SUS": "leitos_uti_adulto_sus",
27         "UTI_PEDIATRICO_SUS": "leitos_uti_pediatico_sus",
28         "UTI_NEONATAL_SUS": "leitos_uti_neonatal_sus",
29     }
30     agregacoes = {coluna_leitos: "sum"}
31     for coluna_original in colunas_uti_especialidade:
32         if coluna_original in df.columns:
33             df[coluna_original] = pd.to_numeric(df[coluna_original], errors="coerce")
34             agregacoes[coluna_original] = "sum"
35         else:
36             print(f"Aviso: coluna '{coluna_original}' não encontrada - pulando essa especialidade.")
37
38     if coluna_leitos_existentes in df.columns:
39         df[coluna_leitos_existentes] = pd.to_numeric(df[coluna_leitos_existentes], errors="coerce")
40         agregacoes[coluna_leitos_existentes] = "sum"
41     else:
42         print(f"Aviso: coluna '{coluna_leitos_existentes}' não encontrada - não será possível calcular proporcao_leitos_sus.")
43
44     ocupacao = df.groupby("municipio").agg(agregacoes).reset_index()
45     ocupacao = ocupacao.rename(columns=colunas_uti_especialidade)
46
47     if coluna_leitos_existentes in ocupacao.columns:
48         ocupacao["proporcao_leitos_sus"] = ocupacao[coluna_leitos] / ocupacao[coluna_leitos_existentes]
49
50     if coluna_cnes in df.columns:
51         hospitais_sus = (
52             df[df[coluna_leitos] > 0]
53             .groupby("municipio")[coluna_cnes]
54             .nunique()
55             .reset_index(name="n_hospitais_sus")
56         )
57         ocupacao = ocupacao.merge(hospitais_sus, on="municipio", how="left")
58         ocupacao["n_hospitais_sus"] = ocupacao["n_hospitais_sus"].fillna(0)
59     else:
60         print(f"Aviso: coluna '{coluna_cnes}' não encontrada - não será possível calcular n_hospitais_sus.")
61
62     print("Leitos: OK")
63     return ocupacao
64
```


Etapa 4 — População (IBGE)

Função que lê o arquivo de população do IBGE — que vem num formato aninhado, como a API do IBGE entrega — e extrai só o que interessa: o nome do município e sua população em 2025.

Etapa 4 — População (IBGE)

O JSON vem no formato aninhado da API do IBGE (SIDRA).

```
[10]
```

```
✓ Os
```

```
1 def carregar_populacao():
2     caminho = os.path.join(PASTA_DADOS, "populacao_municipios_2025 (1).json")
3     print("Carregando População...")
4     with open(caminho, encoding="utf-8") as f:
5         dados = json.load(f)
6
7     series = dados[0]["resultados"][0]["series"]
8
9     registros = []
10    for item in series:
11        nome_completo = item["localidade"]["nome"]
12        populacao = item["serie"].get("2025")
13        if " - " in nome_completo:
14            municipio, uf = nome_completo.rsplit(" - ", 1)
15        else:
16            municipio, uf = nome_completo, None
17        registros.append({
18            "municipio": normalizar_municipio(municipio),
19            "uf": uf,
20            "populacao": int(populacao) if populacao is not None else None,
21        })
22
23    populacao_df = pd.DataFrame(registros)
24    populacao_df = populacao_df[populacao_df["uf"] == UF_ALVO]
25    print("População: OK")
26    return populacao_df[["municipio", "populacao"]]
27
```

Classificação regional

Função que usa a coordenada geográfica de cada município (calculada a partir do centro do estado) para classificá-lo automaticamente em Norte, Sul, Leste, Oeste ou Centro.

Classificação regional (Norte / Sul / Leste / Oeste / Centro)

Reaproveita a geocodificação já feita para a imputação de temperatura — cada município é classificado com base na posição da sua coordenada em relação ao **centro geográfico do estado** (mediana das coordenadas de todos os municípios do dataset), não em relação à capital.

Como funciona: calcula o centro (mediana de latitude/longitude) e uma faixa central em torno dele. Município dentro dessa faixa nos dois eixos vira "Centro". Fora da faixa, compara se o desvio é maior em latitude (Norte/Sul) ou em longitude (Leste/Oeste) e classifica pelo eixo dominante. Município sem coordenada encontrada vira "Não informado".

```
[11]
✓ Os
1 def geocodificar_municipios(lista_municipios, pausa_segundos=1.0):
2     """Descobre a latitude/longitude de cada município via OpenStreetMap
3     (Nominatim). Respeita 1 requisição por segundo (limite do serviço gratuito)."""
4     from geopy.geocoders import Nominatim
5     geolocator = Nominatim(user_agent="vigiar_etl_fiap")
6     coordenadas = {}
7     for i, municipio in enumerate(lista_municipios, start=1):
8         try:
9             local = geolocator.geocode(f"{municipio}, São Paulo, Brasil", timeout=10)
10            coordenadas[municipio] = {"lat": local.latitude, "lon": local.longitude} if local else {"lat": None, "lon": None}
11        except Exception:
12            coordenadas[municipio] = {"lat": None, "lon": None}
13            time.sleep(pausa_segundos)
14            if i % 50 == 0:
15                print(f" {i}/{len(lista_municipios)} municípios geocodificados")
16    return coordenadas
17
18
19 def classificar_regiao(municipios_coords):
20     """Classifica cada município em Norte/Sul/Leste/Oeste/Centro, com base na
21     posição em relação à mediana das coordenadas de todos os municípios (o
22     'centro geográfico' do conjunto de dados)."""
23     lats = [c["lat"] for c in municipios_coords.values() if c["lat"] is not None]
24     lons = [c["lon"] for c in municipios_coords.values() if c["lon"] is not None]
25     if not lats or not lons:
26         return {m: "Não informado" for m in municipios_coords}
27
28     lat_central, lon_central = np.median(lats), np.median(lons)
29     margem_lat = (max(lats) - min(lats)) / 6 or 0.01
30     margem_lon = (max(lons) - min(lons)) / 6 or 0.01
31
32     regioes = {}
33     for municipio, coord in municipios_coords.items():
34         if coord["lat"] is None or coord["lon"] is None:
35             regioes[municipio] = "Não informado"
36             continue
37         delta_lat = coord["lat"] - lat_central
38         delta_lon = coord["lon"] - lon_central
39         if abs(delta_lat) < margem_lat and abs(delta_lon) < margem_lon:
40             regioes[municipio] = "Centro"
41         elif abs(delta_lat) / margem_lat >= abs(delta_lon) / margem_lon:
42             regioes[municipio] = "Norte" if delta_lat > 0 else "Sul"
43         else:
44             regioes[municipio] = "Leste" if delta_lon > 0 else "Oeste"
45    return regioes
46
```

Montagem da grade completa

Função que garante que todo município tenha uma linha em toda semana do período analisado — mesmo que não tenha havido nenhum caso registrado naquela semana, o valor vira zero em vez de a linha simplesmente não existir. Em seguida, junta as quatro fontes de dados numa tabela só.

Grade completa de município x semana + união final

Antes de unir, criamos a **grade completa** de município x semana — todo município tem uma linha em toda semana do período, mesmo sem notificação de SRAG (nesse caso, contagens viram 0, em vez de a linha simplesmente não existir). População, leitos e temperatura continuam podendo ficar vazios quando não há dado real disponível.

```
[12]
✓ Os
1 def completar_grade_e_unir(srag, populacao, leitos, temperatura):
2     # Filtro adicionado após investigação: como "município" agora vem do município de
3     # RESIDÊNCIA (ID_MN_RESI), pacientes que moram em outro estado mas foram atendidos em
4     # SP (filtro de UF é sobre a notificação, não a residência) entravam na base. Isso não
5     # só inflava o dataset com municípios fora de escopo – contaminava a geocodificação e
6     # a classificação regional: max(lat)/min(lat) passava a medir a extensão do BRASIL
7     # inteiro em vez de só SP, e a margem de "Centro" ficava tão grande que 91% dos
8     # municípios paulistas caíam em "Centro" e Norte/Sul ficavam vazios. Filtrando aqui,
9     # antes de montar todos_municipios, o problema é resolvido na raiz – tanto pro dataset
10    # final quanto pra geocodificação/classificação regional que usa essa mesma lista.
11    municipios_sp = set(populacao["municipio"])
12    n_municipios_antes = srag["municipio"].nunique()
13    n_linhas_antes = len(srag)
14    srag = srag[srag["municipio"].isin(municipios_sp)].copy()
15    n_municipios_removidos = n_municipios_antes - srag["municipio"].nunique()
16    n_linhas_removidas = n_linhas_antes - len(srag)
17    if n_municipios_removidos > 0:
18        print(f"SRAG: removidos {n_municipios_removidos} municípios ({n_linhas_removidas} linhas) "
19              f"sem correspondência na tabela de população – residência fora de SP ou não informada.")
20
21    todos_municipios = pd.concat([
22        srag["municipio"], populacao["municipio"], leitos["municipio"]
23    ]).unique()
24    semanas = pd.period_range(srag["semana"].min(), srag["semana"].max(), freq="W")
25
26    grade = pd.MultiIndex.from_product(
27        [todos_municipios, semanas], names=["municipio", "semana"]
28    ).to_frame(index=False)
29
30    dataset = grade.merge(srag, on=["municipio", "semana"], how="left")
31    dataset = dataset.merge(populacao, on="municipio", how="left")
32    dataset = dataset.merge(leitos, on="municipio", how="left")
33    dataset = dataset.merge(temperatura, on=["municipio", "semana"], how="left")
34
35    colunas_contagem = [
36        "casos_srag", "casos_influenza", "casos_covid", "obitos",
37        "casos_uti", "casos_com_comorbidade", "casos_vacinados", "casos_idosos",
38        "taxa_obito", "taxa_uti", "taxa_comorbidade", "taxa_vacinados", "taxa_idosos",
39        # novas variáveis: contagens e taxas (0 é o valor correto quando não há
40        # casos na semana). As colunas de dias (tempo_medio_uti dias,
41        # atraso_* dias, tempo_ate_desfecho_dias) ficam de fora de propósito –
42        # continuam NaN quando não há caso, porque "0 dias de atraso" seria
43        # uma informação errada, não "não houve caso".
44        "casos_surto", "casos_saturacao_baixa", "casos_dispneia", "casos_desc_resp",
45        "casos_vent_invasiva", "casos_pcr_positivo", "casos_antigenico_positivo",
46        "casos_sars2", "casos_vsr", "casos_testados", "casos_fator_risco",
47        "casos_gestantes", "casos_tabagistas", "casos_zona_rural",
48        "casos_vacinados_covid", "casos_febre", "casos_tosse", "casos_referenciados",
49        "taxa_cadeia_surto", "taxa_saturacao_baixa", "taxa_dispneia",
50        "taxa_desc_respiratorio", "taxa_ventilacao_invasiva",
51        "taxa_positividade_pcr", "taxa_positividade_antigenico", "taxa_sars2",
52        "taxa_vsr", "taxa_testagem", "taxa_fator_risco", "taxa_gestantes",
53        "taxa_tabagismo", "taxa_zona_rural", "taxa_vacinacao_covid",
54        "taxa_febre", "taxa_tosse", "indice_fluxo_referencia_hospitalar",
55    ]
56    for coluna in colunas_contagem:
57        if coluna in dataset.columns:
58            dataset[coluna] = dataset[coluna].fillna(0)
59
60    return dataset, todos_municipios
61
```

Geração do dataset final — início

Função que preenche a temperatura dos municípios sem estação meteorológica própria, usando a temperatura da estação mais próxima (calculada por distância geográfica).

Geração do dataset final

Une tudo, geocodifica os municípios (uma única vez, reaproveitada para região e temperatura), imputa temperatura ausente por proximidade, calcula colunas derivadas e formata as datas.

```

13] 1 def preencher_temperatura_por_proximidade(dataset, temperatura, estacoes_coords, municipios_coords):
19min 2     """Para linhas sem temp_media, usa a temperatura da estação mais próxima
3     do município (calculada por coordenada), marcando a linha em
4     'temp_imputada'."""
5     dataset = dataset.copy()
6     dataset["temp_imputada"] = dataset["temp_media"].isna()
7
8     def estacao_mais_proxima(lat, lon):
9         melhor, menor_dist = None, float("inf")
10        for nome_estacao, info in estacoes_coords.items():
11            if info["lat"] is None or info["lon"] is None:
12                continue
13            R = 6371
14            dlat, dlon = radians(info["lat"] - lat), radians(info["lon"] - lon)
15            a = sin(dlat / 2) ** 2 + cos(radians(lat)) * cos(radians(info["lat"])) * sin(dlon / 2) ** 2
16            dist = R * 2 * atan2(sqrt(a), sqrt(1 - a))
17            if dist < menor_dist:
18                menor_dist, melhor = dist, nome_estacao
19        return melhor
20
21    mapa_estacao_mais_proxima = {}
22    for municipio in dataset.loc[dataset["temp_imputada"], "municipio"].unique():
23        coord = municipios_coords.get(municipio, {"lat": None, "lon": None})
24        if coord["lat"] is None:
25            mapa_estacao_mais_proxima[municipio] = None
26        else:
27            mapa_estacao_mais_proxima[municipio] = estacao_mais_proxima(coord["lat"], coord["lon"])
28
29    def buscar_temp_proxima(linha):
30        if not linha["temp_imputada"]:
31            return linha["temp_media"]
32        estacao_proxima = mapa_estacao_mais_proxima.get(linha["municipio"])
33        if estacao_proxima is None:
34            return None
35        valores = temperatura.loc[
36            (temperatura["municipio"] == estacao_proxima) & (temperatura["semana"] == linha["semana"]),
37            "temp_media"
38        ]
39        return valores.iloc[0] if len(valores) else None
40
41    dataset["temp_media"] = dataset.apply(buscar_temp_proxima, axis=1)
42    n_imputadas = dataset.loc[dataset["temp_imputada"], "temp_media"].notna().sum()
43    print(f"{n_imputadas} de {dataset['temp_imputada'].sum()} linhas preenchidas por proximidade.")
44    return dataset
45
46
47 def gerar_dataset_final():
48     print("Carregando e tratando cada fonte (recorte: São Paulo)...")
49     temperatura, estacoes_coords = carregar_temperatura_sp()
50     srag = carregar_srag()
51     leitos = carregar_leitos()
52     populacao = carregar_populacao()
53
54     print("Completando a grade de município x semana (semanas sem SRAG viram 0)...")
55     dataset, todos_municipios = completar_grade_e_unir(srag, populacao, leitos, temperatura)
56
57     print(f"Geocodificando {len(todos_municipios)} municípios (usado para região + temperatura)...")
58     municipios_coords = geocodificar_municipios(todos_municipios)
59
60     print("Classificando região de cada município...")
61     mapa_regiao = classificar_regiao(municipios_coords)
62     dataset["regiao"] = dataset["municipio"].map(mapa_regiao).fillna("Não informado")

```


Geração do dataset final — continuação

Cálculo das últimas colunas derivadas (casos a cada 10 mil habitantes, leitos a cada mil habitantes, média móvel de 3 semanas, variação percentual de casos) e o salvamento do arquivo CSV final com todas as colunas organizadas.

```

63
64 print("Preenchendo temperaturas ausentes por proximidade geográfica...")
65 dataset = preencher_temperatura_por_proximidade(dataset, temperatura, estacoes_coords, municipios_coords)
66
67 print("Calculando colunas derivadas (sazonalidade, capacidade, tendência)...")
68 dataset = dataset.sort_values(["municipio", "semana"]).reset_index(drop=True)
69
70 dataset["casos_por_10k"] = dataset["casos_srag"] / dataset["populacao"] * 10_000
71 dataset["leitos_por_1000_hab"] = dataset["LEITOS_SUS"] / dataset["populacao"] * 1000
72
73 dataset["data_inicio_semana"] = dataset["semana"].dt.start_time
74 dataset["ano"] = dataset["data_inicio_semana"].dt.year
75 dataset["semana_numero"] = dataset["data_inicio_semana"].dt.isocalendar().week
76 dataset["data_formatada"] = dataset["data_inicio_semana"].apply(formatar_data_br)
77
78 dataset["casos_srag_semana_anterior"] = dataset.groupby("municipio")["casos_srag"].shift(1)
79 dataset["media_movel_3_semanas_casos"] = dataset.groupby("municipio")["casos_srag"].transform(
80     lambda serie: serie.shift(1).rolling(window=3, min_periods=1).mean()
81 )
82 dataset["variacao_temp"] = dataset.groupby("municipio")["temp_media"].diff()
83 dataset["variacao_casos_pct"] = (
84     dataset.groupby("municipio")["casos_srag"].pct_change()
85     .replace([float("inf"), float("-inf")], None)
86 )
87
88 dataset = dataset.drop(columns=["semana", "data_inicio_semana"])
89
90 colunas_finais = [
91     "municipio", "regiao", "regional_saude", "ano", "semana_numero", "data_formatada",
92     "casos_srag", "casos_influenza", "casos_covid", "obitos", "casos_uti",
93     "casos_com_comorbidade", "casos_vacinados", "casos_idosos",
94     "taxa_obito", "taxa_uti", "taxa_comorbidade", "taxa_vacinados", "taxa_idosos",
95     "populacao", "LEITOS_SUS", "leitos_uti_sus", "leitos_uti_adulto_sus",
96     "leitos_uti_pediatico_sus", "leitos_uti_neonatal_sus", "LEITOS_EXISTENTES",
97     "proporcao_leitos_sus", "n_hospitais_sus", "leitos_por_1000_hab",
98     "temp_media", "temp_imputada", "casos_por_10k",
99     "casos_srag_semana_anterior", "media_movel_3_semanas_casos",
100     "variacao_temp", "variacao_casos_pct",
101     # 24 novas variáveis (tabela de features anexada na conversa)
102     "taxa_cadeia_surto", "taxa_saturacao_baixa", "taxa_dispneia",
103     "taxa_desc_respiratorio", "taxa_ventilacao_invasiva", "tempo_medio_uti_dias",
104     "taxa_positividade_pcr", "taxa_positividade_antigenico", "taxa_sars2", "taxa_vsr",
105     "taxa_testagem", "atraso_notificacao_dias", "atraso_coleta_dias",
106     "indice_fluxo_referencia_hospitalar", "atraso_internacao_dias",
107     "taxa_fator_risco", "taxa_gestantes", "taxa_tabagismo", "taxa_zona_rural",
108     "taxa_vacinacao_covid", "taxa_febre", "taxa_tosse",
109     "atraso_digitacao_dias", "tempo_ate_desfecho_dias",
110 ]
111 colunas_finais = [c for c in colunas_finais if c in dataset.columns]
112 dataset = dataset[colunas_finais]
113
114 dataset.to_csv(SAIDA, index=False)
115 print(f"\nDataset final salvo em: {SAIDA}")
116 print(f"Municípios únicos: {dataset['municipio'].nunique()}")
117 print(f"Total de linhas: {len(dataset)}")
118 return dataset
119
120
121 dataset_final = gerar_dataset_final()
122 display(dataset_final.head())
123

```

Execução completa e download do resultado

Log completo da execução: cada etapa (temperatura, SRAG, leitos, população, geocodificação, classificação regional) é concluída com sucesso, o dataset final é salvo com 645 municípios e 54.180 linhas, e uma prévia da tabela final é exibida. Por fim, o código que baixa o CSV para o computador.

Carregando e tratando cada fonte (recorte: São Paulo)...

46 estações de SP encontradas

Temperatura: OK

Carregando SRAG...

Arquivo: srsg_2022_completo.csv não usa ',' como separador - detectando automaticamente...

/tmp/ipykernel_1697/3836132284.py:24: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.

df["semana"] = df["data_notificacao"].dt.to_period("W")

Análise de valores em ID_MUN_RES: ['SAO PAULO', 'SAO PAULO', 'SAO PAULO', 'GUARULHOS', 'SAO VICENTE']

SRAG: OK

Carregando Leitos...

Leitos: OK

Carregando População...

População: OK

Completando a grade de município x semana (semanas sem SRAG viram 0)...

SRAG: removidos 123 municípios (203 linhas) sem correspondência na tabela de população - residência fora de SP ou não informada.

/tmp/ipykernel_1697/1361943402.py:58: FutureWarning: Downcasting object dtype arrays on .fillna, .ffill, .bfill is deprecated and will change in a future version. Call result.infer_objects(copy=False) instead. To opt-in to the future behavior, set 'pd.set_option('future.no_silent_downcasting', True)'

dataset[column] = dataset[column].fillna(0)

Geocodificando 645 municípios (usado para região + temperatura)...

50/645 municípios geocodificados

100/645 municípios geocodificados

150/645 municípios geocodificados

200/645 municípios geocodificados

250/645 municípios geocodificados

300/645 municípios geocodificados

350/645 municípios geocodificados

400/645 municípios geocodificados

450/645 municípios geocodificados

500/645 municípios geocodificados

550/645 municípios geocodificados

600/645 municípios geocodificados

Classificando região de cada município...

Preenchendo temperaturas ausentes por proximidade geográfica...

23221 de 52300 linhas preenchidas por proximidade.

Calculando colunas derivadas (sazonalidade, capacidade, tendência)...

Dataset final salvo em: /content/dataset_vigiar_sp.csv

Municípios únicos: 645

Total de linhas: 54180

	município	região	regional_saude	ano	semana_numero	data_formatada	casos_srsg	casos_influenza	casos_covid	obitos	...	atraso_internacao_dias	taxa_fator_risco	taxa_gestantes	taxa_tabagismo	taxa_zona_rural	taxa_vacinacao_covid	taxa_febre	taxa_tosse	atraso_digitacao_dias	tempo_ate_desfecho_dias
0	ADAMANTINA	Oeste	NaN	2024	52	23/12/2024	0.0	0.0	0.0	0.0	...	NaN	0.0	0.0	0.0	0.0	0.0	0.0	0.0	NaN	NaN
1	ADAMANTINA	Oeste	NaN	2024	1	30/12/2024	0.0	0.0	0.0	0.0	...	NaN	0.0	0.0	0.0	0.0	0.0	0.0	0.0	NaN	NaN
2	ADAMANTINA	Oeste	NaN	2025	2	06/01/2025	0.0	0.0	0.0	0.0	...	NaN	0.0	0.0	0.0	0.0	0.0	0.0	0.0	NaN	NaN
3	ADAMANTINA	Oeste	NaN	2025	3	13/01/2025	0.0	0.0	0.0	0.0	...	NaN	0.0	0.0	0.0	0.0	0.0	0.0	0.0	NaN	NaN
4	ADAMANTINA	Oeste	NaN	2025	4	20/01/2025	0.0	0.0	0.0	0.0	...	NaN	0.0	0.0	0.0	0.0	0.0	0.0	0.0	NaN	NaN

5 rows × 60 columns

Download do resultado

1 from google.colab import files

2

3 files.download(SAIDA)

4

5. Apêndice — código-fonte completo

Código completo do arquivo `etl_vigiar_colab_v3.py`, estruturado e em funcionamento, conforme executado nas evidências da seção anterior.

```
1  # -*- coding: utf-8 -*-
2  """etl_vigiar_colab_v3 (2).ipynb
3
4  Automatically generated by Colab.
5
6  Original file is located at
7      https://colab.research.google.com/drive/1sUGkieNxVH8pGB0tklqnFuzSJEB5c_q
8
9  # ETL - Vigiar (v3 - granularidade semanal, região, decodificação de campos)
10
11  Pipeline completo: baixa os dados do bucket Oracle Cloud, trata cada fonte (SRAG, Leitos, Temperatura,
12  População), une tudo num único CSV - recorte São Paulo, agregado por **semana** (não mais por mês).
13
14  **Novidades desta versão em relação à anterior:**
15  - Dados no formato brasileiro (ex: '29/08/2025').
16  - Campos categóricos do SRAG (ex: 'EVOLUCAO') decodificados para o significado real, usando o
17  dicionário de dados oficial do SIVEP-Gripe - em vez do código numérico cru.
18  - Agregação por **semana** em vez de mês.
19  - Nova coluna 'regiao' (Norte/Sul/Leste/Oeste/Centro), com base na posição geográfica do município
20  dentro do estado.
21  - Mantida a imputação de temperatura por proximidade geográfica para municípios sem estação do INMET.
22
23  **Como usar:** rode as células de cima pra baixo. A célula de download pula arquivos já existentes,
24  então é seguro rodar de novo se a sessão reiniciar no meio do processo.
25
26  ## Download dos dados brutos
27  """
28
29  # (Opcional, mas recomendado) Monta o Google Drive.
30  # Sem isso, os arquivos baixados somem toda vez que a sessão do Colab reinicia
31  # por inatividade e você precisa baixar tudo de novo.
32
33  # from google.colab import drive
34  # drive.mount('/content/drive')
35
36  import os
37  import requests
38  from urllib.parse import quote
39
40  BASE_URL = (
41      "https://objectstorage.sa-saopaulo-1.oraclecloud.com/p/"
42      "3RgzWpKLSx2idlorDnZng95zcleF-EaBwY30lePGCOuqIwT_y2GpXJHhmKoOw00/"
43      "n/grksbwp7ro2x/b/teste_vigiar/o/"
44  )
45  PASTA_DADOS = "/content/downloads" # AJUSTAR para "/content/drive/MyDrive/..." se usar o Drive
46
47
48  def baixar_todos_os_arquivos():
49      resp = requests.get(BASE_URL)
50      resp.raise_for_status()
51      objetos = [obj["name"] for obj in resp.json()["objects"]]
52      print(f"{len(objetos)} objetos encontrados no bucket. Baixando...")
53
54      for i, nome in enumerate(objetos, start=1):
55          if nome.endswith("/"):
56              continue
57          caminho_local = os.path.join(PASTA_DADOS, nome)
58          os.makedirs(os.path.dirname(caminho_local), exist_ok=True)
59          if os.path.exists(caminho_local):
60              continue
61          with requests.get(BASE_URL + quote(nome), stream=True) as r:
62              r.raise_for_status()
63              with open(caminho_local, "wb") as f:
64                  for chunk in r.iter_content(chunk_size=8192):
65                      f.write(chunk)
66                  if i % 25 == 0 or i == len(objetos):
67                      print(f" {i}/{len(objetos)} processados")
68
69      print("Download concluído. Arquivos em:", PASTA_DADOS)
70
71  baixar_todos_os_arquivos()
72
73  """## Configuração
74
75  Define constantes, funções auxiliares de leitura, a normalização de nomes de município e os
76  dicionários de decodificação dos campos categóricos do SRAG (baseados no dicionário de dados oficial
77  do SIVEP-Gripe).
78  """
79
80  import glob
81  import json
82  import math
83  import time
84  import unicodedata
85  from math import radians, sin, cos, sqrt, atan2
86
87  import numpy as np
88  import pandas as pd
89
90  UF_ALVO = "SP" # recorte do projeto: só dados do estado de São Paulo
91
92  # Municípios cujo nome oficial mudou ou difere entre as bases, sem ser só uma
93  # questão de acentuação (a normalização abaixo não resolve esses casos).
```

```

96 EXCECOES_NOME_MUNICIPIO = {
97     "MOJI MIRIM": "MOGI MIRIM",
98     "SAO LUIS DO PARAITINGA": "SAO LUIZ DO PARAITINGA",
99     "BIRITIBA MIRIM": "BIRITIBA-MIRIM",
100 }
101
102
103 def normalizar_municipio(texto):
104     """Deixa o nome do município em maiúsculo, sem espaços nas pontas e sem
105     acentuação, e corrige um pequeno número de nomes com grafia divergente
106     entre as bases."""
107     texto = str(texto).upper().strip()
108     texto = unicodedata.normalize("NFKD", texto).encode("ASCII", "ignore").decode("ASCII")
109     texto = EXCECOES_NOME_MUNICIPIO.get(texto, texto)
110     return texto
111
112
113 def formatar_data_br(data):
114     """Converte uma data/timestamp do pandas para o formato brasileiro
115     DD/MM/AAAA. Retorna 'Não informado' se a data for nula."""
116     if pd.isna(data):
117         return "Não informado"
118     return pd.Timestamp(data).strftime("%d/%m/%Y")
119
120
121 def ler_csv_seguro(caminho, **kwargs):
122     """Lê um CSV tentando UTF-8 e depois Latin-1, com separador padrão
123     (vírgula). Se der erro de formatação — comum quando o arquivo na verdade
124     usa ';' como separador — refaz a leitura deixando o pandas detectar o
125     separador automaticamente."""
126     for encoding in ("utf-8", "latin1"):
127         try:
128             df = pd.read_csv(caminho, encoding=encoding, **kwargs)
129             if df.shape[1] <= 1:
130                 raise pd.errors.ParserError("Apenas 1 coluna detectada")
131             return df
132         except UnicodeDecodeError:
133             continue
134     except pd.errors.ParserError:
135         print(f"Aviso: {os.path.basename(caminho)} não usa ',' como separador — detectando automaticamente...")
136         return pd.read_csv(caminho, encoding=encoding, sep=None, engine="python", on_bad_lines="skip", **kwargs)
137     raise UnicodeDecodeError(f"Não foi possível ler {caminho} nem com utf-8 nem com latin1.")
138
139
140 # — Dicionários de decodificação (dicionário de dados oficial SIVEP-Gripe) —
141 # Aplicados linha a linha, ANTES da agregação por semana/município — o CSV
142 # final continua com contagens/taxas numéricas, só que calculadas em cima do
143 # significado real do campo, não do código cru. Isso deixa o processo mais
144 # auditável (fica claro no código o que cada valor representa) e evita erro
145 # de interpretação de código.
146
147 MAPA_SIM_NAO = {1: "Sim", 2: "Não", 9: "Ignorado"}
148 MAPA_UTI = {1: "Sim", 2: "Não", 9: "Ignorado"}
149 MAPA_VACINA = {1: "Sim", 2: "Não", 9: "Ignorado"}
150 MAPA_TP_IDADE = {1: "Dia", 2: "Mês", 3: "Ano"}
151 MAPA_EVOLUCAO = {1: "Cura", 2: "Óbito", 3: "Óbito por outras causas", 9: "Ignorado"}
152 MAPA_CLASSI_FIN = {
153     1: "SRAG por influenza",
154     2: "SRAG por outro vírus respiratório",
155     3: "SRAG por outro agente etiológico",
156     4: "SRAG não especificado",
157     5: "SRAG por covid-19",
158 }
159
160 MAPA_SUPORT_VEN = {1: "Sim, invasivo", 2: "Sim, não invasivo", 3: "Não", 9: "Ignorado"}
161 MAPA_PCR_RESUL = {1: "Detectável", 2: "Não detectável", 3: "Inconclusivo",
162     4: "Não realizado", 5: "Aguardando resultado", 9: "Ignorado"}
163
164 MAPA_RES_AN = {1: "Positivo", 2: "Negativo", 3: "Inconclusivo",
165     4: "Não realizado", 5: "Aguardando resultado", 9: "Ignorado"}
166
167 MAPA_CS_GESTANT = {1: "1º trimestre", 2: "2º trimestre", 3: "3º trimestre",
168     4: "Idade gestacional ignorada", 5: "Não", 6: "Não se aplica", 9: "Ignorado"}
169
170 MAPA_CS_ZONA = {1: "Urbana", 2: "Rural", 3: "Periurbana", 9: "Ignorado"}
171
172
173 def decodificar_categoria(serie, mapa):
174     """Converte uma coluna de códigos numéricos (ex: 1, 2, 9) para o
175     significado real (ex: 'Sim', 'Não', 'Ignorado'), usando o dicionário de
176     dados oficial. Valores ausentes ou códigos fora do mapa viram
177     'Não informado', em vez de ficar em branco."""
178     numerico = pd.to_numeric(serie, errors="coerce")
179     return numerico.map(mapa).fillna("Não informado")
180
181
182 def serie_segura(df, coluna):
183     """Devolve a coluna se ela existir no DataFrame; senão devolve uma coluna
184     vazia (tudo NaN) e avisa — assim uma variável faltando não derruba o
185     pipeline inteiro, só fica em branco no resultado final."""
186     if coluna not in df.columns:
187         print(f"Aviso: coluna '{coluna}' não encontrada no CSV — as variáveis derivadas dela ficarão vazias.")
188         return pd.Series([None] * len(df), index=df.index)
189     return df[coluna]
190
191
192 SAIDA = "/content/dataset_vigiar_sp.csv" # ajuste para /content/drive/... se estiver usando o Drive
193
194
195 """## Etapa 0 — Inspeção
196
197 Confere a estrutura real de cada arquivo antes de tratar qualquer coisa.
198 """
199
200 def encontrar_linha_cabecalho(caminho, encoding="latin1"):
201     """Procura a linha cujo PRIMEIRO campo (antes do ';') é exatamente 'Data'.
202     Não basta checar se a linha começa com 'Data', porque arquivos do INMET têm
203     uma linha de metadado 'DATA DE FUNDACAO;...' que também começaria com
204     'Data' e seria confundida com o cabeçalho real da tabela."""
205     with open(caminho, encoding=encoding) as f:
206         for i, linha in enumerate(f):
207             primeiro_campo = linha.split(";")[0].strip().upper()
208             if primeiro_campo == "DATA":
209                 return i
210     return 8
211
212
213 def extrair_coordenadas_estacao(caminho, encoding="latin1"):

```

```

208 """Iê a latitude/longitude da estação, presentes no bloco de metadados
209 no topo do arquivo."""
210 lat, lon = None, None
211 with open(caminho, encoding=encoding) as f:
212     for linha in f:
213         primeiro_campo = linha.split(";")[0].strip().upper()
214         if primeiro_campo.startswith("LATITUDE"):
215             try:
216                 lat = float(linha.split(";")[1].strip().replace(",", "."))
217             except (IndexError, ValueError):
218                 pass
219         elif primeiro_campo.startswith("LONGITUDE"):
220             try:
221                 lon = float(linha.split(";")[1].strip().replace(",", "."))
222             except (IndexError, ValueError):
223                 pass
224         if primeiro_campo == "DATA":
225             break
226     return lat, lon
227
228
229 def encontrar_coluna(colunas, texto_procurado):
230     """Retorna a primeira coluna cujo nome contém o texto procurado (case-insensitive)."""
231     texto_procurado = texto_procurado.upper()
232     for c in colunas:
233         if texto_procurado in str(c).upper():
234             return c
235     return None
236
237
238 def inspecionar_csv(caminho, n_linhas=10, **kwargs):
239     print(f"\n{'*' * 70}\n{caminho}\n{'*' * 70}")
240     df = ler_csv_seguro(caminho, nrows=n_linhas, **kwargs)
241     print("Colunas:", list(df.columns))
242     display(df.head())
243
244
245 def inspecionar_json(caminho):
246     print(f"\n{'*' * 70}\n{caminho}\n{'*' * 70}")
247     with open(caminho, encoding="utf-8") as f:
248         dados = json.load(f)
249     amostra = dados[:3] if isinstance(dados, list) else dados
250     print(json.dumps(amostra, indent=2, ensure_ascii=False)[:1500])
251
252
253 def inspecionar_inmet(caminho, n_linhas=15):
254     print(f"\n{'*' * 70}\n{caminho}\n(bruto, primeiras linhas)\n{'*' * 70}")
255     with open(caminho, encoding="latin1") as f:
256         for i, linha in enumerate(f):
257             print(linha.strip())
258             if i >= n_linhas:
259                 break
260
261
262 def inspecionar_tudo():
263     inspecionar_csv(os.path.join(PASTA_DADOS, "Leitos_2025.csv"))
264     inspecionar_csv(os.path.join(PASTA_DADOS, "srag_2025_completo.csv"))
265     inspecionar_json(os.path.join(PASTA_DADOS, "cnes_estabelecimentos.json"))
266     inspecionar_json(os.path.join(PASTA_DADOS, "populacao_municipios_2025 (1).json"))
267     exemplos_sp = glob.glob(os.path.join(PASTA_DADOS, "TEMPERATURA", "INMET_SE_SP_*.CSV"))
268     if exemplos_sp:
269         inspecionar_inmet(exemplos_sp[0])
270     else:
271         print("Não encontrei CSVs de temperatura de SP em", os.path.join(PASTA_DADOS, "TEMPERATURA"))
272
273 inspecionar_tudo()
274
275 """## Etapa 1 – Temperatura (INMET, agregada por semana)
276
277 Mesma leitura robusta de antes (detecção automática de cabeçalho, colunas de data/temperatura),
278 mas agora agregada por **semana** em vez de mês. Também extrai a latitude/longitude de cada estação –
279 usadas mais adiante tanto para a imputação de temperatura quanto para a classificação regional dos
280 municípios.
281 """
282
283 def carregar_temperatura_sp():
284     arquivos = glob.glob(os.path.join(PASTA_DADOS, "TEMPERATURA", "INMET_SE_SP_*.CSV"))
285     print(f"{len(arquivos)} estações de SP encontradas")
286
287     tabelas = []
288     falhas = []
289     estacoes_coords = {}
290     for caminho in arquivos:
291         nome_arquivo = os.path.basename(caminho)
292         try:
293             partes = nome_arquivo.replace(".CSV", "").split("_")
294             codigo_estacao = partes[3]
295             cidade_estacao = normalizar_municipio(partes[4].split(" - ")[0])
296
297             lat, lon = extrair_coordenadas_estacao(caminho)
298             estacoes_coords[cidade_estacao] = {"lat": lat, "lon": lon}
299
300             linha_cabecalho = encontrar_linha_cabecalho(caminho)
301
302             df = pd.read_csv(
303                 caminho, sep=";", decimal=".", skiprows=linha_cabecalho,
304                 encoding="latin1", engine="python", on_bad_lines="skip",
305             )
306             df["codigo_estacao"] = codigo_estacao
307             df["cidade_estacao"] = cidade_estacao
308             tabelas.append(df)
309         except Exception as e:
310             falhas.append((nome_arquivo, str(e)))
311
312     if falhas:
313         print(f"\n{len(falhas)} arquivo(s) não puderam ser lidos e foram pulados:")
314         for nome, erro in falhas:
315             print(f" - {nome}: {erro}")
316
317     temperatura = pd.concat(tabelas, ignore_index=True)
318
319     coluna_data_real = encontrar_coluna(temperatura.columns, "DATA")

```

```

320 coluna_temp = encontrar_coluna(temperatura.columns, "TEMPERATURA DO AR")
321
322 if coluna_data_real is None or coluna_temp is None:
323     print("Não encontrei automaticamente as colunas de data/temperatura.")
324     print("Colunas disponíveis:", list(temperatura.columns))
325     raise KeyError("Ajuste coluna_data_real / coluna_temp manualmente com um dos nomes acima.")
326
327 temperatura["data"] = pd.to_datetime(temperatura[coluna_data_real], errors="coerce")
328 temperatura["semana"] = temperatura["data"].dt.to_period("W")
329
330 resumo_semanal = (
331     temperatura
332     .groupby(["cidade_estacao", "semana"])[coluna_temp]
333     .mean()
334     .reset_index()
335     .rename(columns={coluna_temp: "temp_media", "cidade_estacao": "municipio"})
336 )
337 print("Temperatura: OK")
338 return resumo_semanal, estacoes_coords
339
340 """## Etapa 2 — SRAG (base principal, filtrada para SP, agregada por semana)
341
342 Os campos categóricos ('CLASSI_FIN', 'EVOLUCAO', 'UTI', 'VACINA', comorbidades, 'TP_IDADE') são
343 decodificados para o significado real antes de qualquer contagem — usando os dicionários definidos
344 na Configuração. Códigos ausentes ou fora do padrão viram 'Não informado'. As contagens e taxas
345 finais ('casos_influenza', 'taxa_obito', etc.) são calculadas em cima desses valores decodificados,
346 não dos códigos numéricos crus.
347
348 Também extraímos 'regional_saude' (campo 'ID_REGIONA') — a Regional de Saúde oficial que o SUS usa para organizar referência hospitalar entre
municipios, diferente da classificação geográfica simples ('regiao': Norte/Sul/Leste/Oeste/Centro) calculada mais adiante.
349 """
350
351 def carregar_srag():
352     print("Carregando SRAG...")
353     df = ler_csv_seguro(os.path.join(PASTA_DADOS, "srag_2025_completo.csv"))
354
355     coluna_uf = "SG_UF_NOT"
356     coluna_data = "DT_NOTIFIC"
357     coluna_municipio_resi = "ID_MN_RESI" # município de RESIDÊNCIA do paciente (chave geográfica)
358     coluna_municipio_notif = "ID_MUNICIP" # município de NOTIFICAÇÃO (mantido só de referência)
359     coluna_regional = "ID_REGIONA" # Regional de Saúde de notificação (agrupamento oficial do SUS)
360
361     for nome_var, coluna in [
362         ("coluna_uf", coluna_uf), ("coluna_data", coluna_data),
363         ("coluna_municipio_resi", coluna_municipio_resi),
364         ("coluna_municipio_notif", coluna_municipio_notif),
365     ]:
366         if coluna not in df.columns:
367             print(f"\nA coluna '{coluna}' (usada como {nome_var}) não existe no arquivo.")
368             print("Colunas disponíveis:", list(df.columns))
369             raise KeyError(f"Ajuste {nome_var} em carregar_srag() com um dos nomes acima.")
370
371     df = df[df[coluna_uf] == UF_ALVO]
372
373     df["data_notificacao"] = pd.to_datetime(df[coluna_data], errors="coerce")
374     df["semana"] = df["data_notificacao"].dt.to_period("W")
375
376     # Checagem: ID_MN_RESI precisa vir como NOME de município (texto), igual
377     # ID_MUNICIP, pra normalizar_municipio() funcionar. Se vier como código
378     # numérico do IBGE, o aviso abaixo aparece — nesse caso normalizar_municipio
379     # não resolve sozinho, precisaria de uma tabela código -> nome antes.
380     amostra = df[coluna_municipio_resi].dropna().astype(str).head(5).tolist()
381     print(f"Amostra de valores em {coluna_municipio_resi}: {amostra}")
382     if amostra and all(v.replace(".", "").isdigit() for v in amostra):
383         print(f"ATENÇÃO: '{coluna_municipio_resi}' parece conter CÓDIGOS numéricos do IBGE, não "
384               f"nomes de município — normalizar_municipio() não vai funcionar direito nesse caso.")
385
386     df["municipio"] = df[coluna_municipio_resi].apply(normalizar_municipio) # chave = residência
387     df["municipio_notificacao"] = df[coluna_municipio_notif].apply(normalizar_municipio) # mantido pra comparação
388
389     if coluna_regional in df.columns:
390         df["regional_saude"] = df[coluna_regional].fillna("Não informado")
391     else:
392         print(f"Aviso: coluna '{coluna_regional}' não encontrada — regional_saude ficará como 'Não informado'.")
393         df["regional_saude"] = "Não informado"
394
395     # — Decodificação dos campos categóricos (dicionário oficial SIVEP-Gripe) —
396     df["classi_fin_decodificado"] = decodificar_categoria(df["CLASSI_FIN"], MAPA_CLASSI_FIN)
397     df["evolucao_decodificada"] = decodificar_categoria(df["EVOLUCAO"], MAPA_EVOLUCAO)
398     df["uti_decodificado"] = decodificar_categoria(df["UTI"], MAPA_UTI)
399     df["vacina_decodificado"] = decodificar_categoria(df["VACINA"], MAPA_VACINA)
400     df["tp_idade_decodificado"] = decodificar_categoria(df["TP_IDADE"], MAPA_TP_IDADE)
401
402     colunas_comorbidade = [
403         "CARDIOPATI", "DIABETES", "ASMA", "OBESIDADE",
404         "IMUNODEPRE", "RENAL", "PNEUMOPATI", "NEUROLOGIC",
405     ]
406     for coluna in colunas_comorbidade:
407         df[f"{coluna.lower()}_decodificado"] = decodificar_categoria(df[coluna], MAPA_SIM_NAO)
408
409     # — Decodificação dos campos novos (gravidade, testagem, risco, sintomas) —
410     df["surto_decodificado"] = decodificar_categoria(serie_segura(df, "SURTO_SG"), MAPA_SIM_NAO)
411     df["saturacao_decodificada"] = decodificar_categoria(serie_segura(df, "SATURACAO"), MAPA_SIM_NAO)
412     df["dispneia_decodificada"] = decodificar_categoria(serie_segura(df, "DISPNEIA"), MAPA_SIM_NAO)
413     df["desc_resp_decodificado"] = decodificar_categoria(serie_segura(df, "DESC_RESP"), MAPA_SIM_NAO)
414     df["suport_ven_decodificado"] = decodificar_categoria(serie_segura(df, "SUPORT_VEN"), MAPA_SUPORT_VEN)
415     df["pcr_resul_decodificado"] = decodificar_categoria(serie_segura(df, "PCR_RESUL"), MAPA_PCR_RESUL)
416     df["res_an_decodificado"] = decodificar_categoria(serie_segura(df, "RES_AN"), MAPA_RES_AN)
417     df["amostra_decodificada"] = decodificar_categoria(serie_segura(df, "AMOSTRA"), MAPA_SIM_NAO)
418     df["fator_risc_decodificado"] = decodificar_categoria(serie_segura(df, "FATOR_RISC"), MAPA_SIM_NAO)
419     df["cs_gestant_decodificado"] = decodificar_categoria(serie_segura(df, "CS_GESTANT"), MAPA_CS_GESTANT)
420     df["tabag_decodificado"] = decodificar_categoria(serie_segura(df, "TABAG"), MAPA_SIM_NAO)
421     df["cs_zona_decodificada"] = decodificar_categoria(serie_segura(df, "CS_ZONA"), MAPA_CS_ZONA)
422     df["vacina_cov_decodificada"] = decodificar_categoria(serie_segura(df, "VACINA_COV"), MAPA_SIM_NAO)
423     df["febre_decodificada"] = decodificar_categoria(serie_segura(df, "FEBRE"), MAPA_SIM_NAO)
424     df["tosse_decodificada"] = decodificar_categoria(serie_segura(df, "TOSSE"), MAPA_SIM_NAO)
425     df["an_sars2_decodificado"] = decodificar_categoria(serie_segura(df, "AN_SARS2"), MAPA_SIM_NAO)
426     df["pcr_sars2_decodificado"] = decodificar_categoria(serie_segura(df, "PCR_SARS2"), MAPA_SIM_NAO)
427     df["an_vsr_decodificado"] = decodificar_categoria(serie_segura(df, "AN_VSR"), MAPA_SIM_NAO)
428     df["pcr_vsr_decodificado"] = decodificar_categoria(serie_segura(df, "PCR_VSR"), MAPA_SIM_NAO)
429
430     # — Indicadores derivados, calculados a partir dos valores decodificados —

```



```

431 df["eh_influenza"] = df["classi_fin_decodificado"] == "SRAG por influenza"
432 df["eh_covid"] = df["classi_fin_decodificado"] == "SRAG por covid-19"
433 df["eh_obito"] = df["evolucao_decodificada"].isin(["Óbito", "Óbito por outras causas"])
434 df["foi_uti"] = df["uti_decodificado"] == "Sim"
435 df["foi_vacinado"] = df["vacina_decodificado"] == "Sim"
436
437 tem_comorbidade = pd.Series(False, index=df.index)
438 for coluna in colunas_comorbidade:
439     tem_comorbidade = tem_comorbidade | (df[f"{coluna.lower()}_decodificado"] == "Sim")
440 df["tem_comorbidade"] = tem_comorbidade
441
442 # Idade: NU_IDADE_N só representa "anos" quando TP_IDADE decodificado == "Ano"
443 idade_em_anos = pd.to_numeric(df["NU_IDADE_N"], errors="coerce").where(
444     df["tp_idade_decodificado"] == "Ano"
445 )
446 df["eh_idoso"] = idade_em_anos >= 60
447
448 # — Indicadores derivados novos —
449 df["eh_surto"] = df["surto_decodificado"] == "Sim"
450 df["eh_saturacao_baixa"] = df["saturacao_decodificada"] == "Sim"
451 df["eh_dispneia"] = df["dispnea_decodificada"] == "Sim"
452 df["eh_desc_resp"] = df["desc_resp_decodificado"] == "Sim"
453 df["eh_vent_invasiva"] = df["suport_ven_decodificado"] == "Sim, invasivo"
454 df["eh_pcr_positivo"] = df["pcr_resul_decodificado"] == "Detectável"
455 df["eh_antigenico_positivo"] = df["res_an_decodificado"] == "Positivo"
456 # SARS2/VSR: "Sim" se QUALQUER um dos dois testes (antígeno OU PCR) deu positivo
457 df["eh_sars2"] = (df["an_sars2_decodificado"] == "Sim") | (df["pcr_sars2_decodificado"] == "Sim")
458 df["eh_vsr"] = (df["an_vsr_decodificado"] == "Sim") | (df["pcr_vsr_decodificado"] == "Sim")
459 df["foi_testado"] = df["amostra_decodificada"] == "Sim"
460 df["tem_fator_risco"] = df["fator_risc_decodificado"] == "Sim"
461 df["eh_gestante"] = df["cs_gestant_decodificado"].isin(
462     ["1º trimestre", "2º trimestre", "3º trimestre", "Idade gestacional ignorada"]
463 )
464 df["eh_tabagista"] = df["tabag_decodificado"] == "Sim"
465 df["eh_zona_rural"] = df["cs_zona_decodificada"] == "Rural"
466 df["foi_vacinado_covid"] = df["vacina_cov_decodificada"] == "Sim"
467 df["teve_febre"] = df["febre_decodificada"] == "Sim"
468 df["teve_tosse"] = df["tosse_decodificada"] == "Sim"
469
470 # Município de internação, pra medir fluxo de pacientes referenciados pra
471 # fora do próprio município de residência (índice fluxo_referencia_hospitalar)
472 df["municipio_internacao"] = serie_segura(df, "ID_MN_INTE").apply(
473     lambda v: normalizar_municipio(v) if pd.notna(v) else v
474 )
475 df["foi_referenciado"] = (
476     df["municipio_internacao"].notna()
477     & (df["municipio_internacao"] != df["municipio"])
478 )
479
480 # Datas auxiliares, usadas nos cálculos de atraso/tempo abaixo
481 df["dt_sin_pri"] = pd.to_datetime(serie_segura(df, "DT_SIN_PRI"), errors="coerce")
482 df["dt_coleta"] = pd.to_datetime(serie_segura(df, "DT_COLETA"), errors="coerce")
483 df["dt_interna"] = pd.to_datetime(serie_segura(df, "DT_INTERNA"), errors="coerce")
484 df["dt_digita"] = pd.to_datetime(serie_segura(df, "DT_DIGITA"), errors="coerce")
485 df["dt_evolucao"] = pd.to_datetime(serie_segura(df, "DT_EVOLUCA"), errors="coerce")
486 df["dt_entuti"] = pd.to_datetime(serie_segura(df, "DT_ENTUTI"), errors="coerce")
487 df["dt_saikuti"] = pd.to_datetime(serie_segura(df, "DT_SAIDUTI"), errors="coerce")
488
489 df["dias_ate_notificacao"] = (df["data_notificacao"] - df["dt_sin_pri"]).dt.days
490 df["dias_ate_coleta"] = (df["dt_coleta"] - df["dt_sin_pri"]).dt.days
491 df["dias_ate_internacao"] = (df["dt_interna"] - df["dt_sin_pri"]).dt.days
492 df["dias_uti"] = (df["dt_saikuti"] - df["dt_entuti"]).dt.days
493 df["dias_ate_digitacao"] = (df["dt_digita"] - df["data_notificacao"]).dt.days
494 df["dias_ate_desfecho"] = (df["dt_evolucao"] - df["dt_interna"]).dt.days
495
496 agregados = df.groupby(["municipio", "semana"]).agg(
497     regional_saude=("regional_saude", "first"),
498     casos_srag=("municipio", "size"),
499     casos_influenza=("eh_influenza", "sum"),
500     casos_covid=("eh_covid", "sum"),
501     obitos=("eh_obito", "sum"),
502     casos_uti=("foi_uti", "sum"),
503     casos_com_comorbidade=("tem_comorbidade", "sum"),
504     casos_vacinados=("foi_vacinado", "sum"),
505     casos_idosos=("eh_idoso", "sum"),
506     casos_surto=("eh_surto", "sum"),
507     casos_saturacao_baixa=("eh_saturacao_baixa", "sum"),
508     casos_dispneia=("eh_dispneia", "sum"),
509     casos_desc_resp=("eh_desc_resp", "sum"),
510     casos_vent_invasiva=("eh_vent_invasiva", "sum"),
511     tempo_medio_uti_dias=("dias_uti", "mean"),
512     casos_pcr_positivo=("eh_pcr_positivo", "sum"),
513     casos_antigenico_positivo=("eh_antigenico_positivo", "sum"),
514     casos_sars2=("eh_sars2", "sum"),
515     casos_vsr=("eh_vsr", "sum"),
516     casos_testados=("foi_testado", "sum"),
517     atraso_notificacao_dias=("dias_ate_notificacao", "mean"),
518     atraso_coleta_dias=("dias_ate_coleta", "mean"),
519     atraso_internacao_dias=("dias_ate_internacao", "mean"),
520     casos_fator_risco=("tem_fator_risco", "sum"),
521     casos_gestantes=("eh_gestante", "sum"),
522     casos_tabagistas=("eh_tabagista", "sum"),
523     casos_zona_rural=("eh_zona_rural", "sum"),
524     casos_vacinados_covid=("foi_vacinado_covid", "sum"),
525     casos_febre=("teve_febre", "sum"),
526     casos_tosse=("teve_tosse", "sum"),
527     atraso_digitacao_dias=("dias_ate_digitacao", "mean"),
528     tempo_ate_desfecho_dias=("dias_ate_desfecho", "mean"),
529     casos_referenciados=("foi_referenciado", "sum"),
530 ).reset_index()
531
532 agregados["taxa_obito"] = agregados["obitos"] / agregados["casos_srag"]
533 agregados["taxa_uti"] = agregados["casos_uti"] / agregados["casos_srag"]
534 agregados["taxa_comorbidade"] = agregados["casos_com_comorbidade"] / agregados["casos_srag"]
535 agregados["taxa_vacinados"] = agregados["casos_vacinados"] / agregados["casos_srag"]
536 agregados["taxa_idosos"] = agregados["casos_idosos"] / agregados["casos_srag"]
537
538 agregados["taxa_cadeia_surto"] = agregados["casos_surto"] / agregados["casos_srag"]
539 agregados["taxa_saturacao_baixa"] = agregados["casos_saturacao_baixa"] / agregados["casos_srag"]
540 agregados["taxa_dispneia"] = agregados["casos_dispneia"] / agregados["casos_srag"]
541 agregados["taxa_desc_respiratorio"] = agregados["casos_desc_resp"] / agregados["casos_srag"]
542 agregados["taxa_ventilacao_invasiva"] = agregados["casos_vent_invasiva"] / agregados["casos_srag"]

```

```

543 agregados["taxa_positividade_pcr"] = (
544     agregados["casos_pcr_positivo"] / agregados["casos_testados"].replace(0, pd.NA)
545 )
546 agregados["taxa_positividade_antigenico"] = (
547     agregados["casos_antigenico_positivo"] / agregados["casos_testados"].replace(0, pd.NA)
548 )
549 agregados["taxa_sars2"] = agregados["casos_sars2"] / agregados["casos_srag"]
550 agregados["taxa_vsr"] = agregados["casos_vsr"] / agregados["casos_srag"]
551 agregados["taxa_testagem"] = agregados["casos_testados"] / agregados["casos_srag"]
552 agregados["taxa_fator_risco"] = agregados["casos_fator_risco"] / agregados["casos_srag"]
553 agregados["taxa_gestantes"] = agregados["casos_gestantes"] / agregados["casos_srag"]
554 agregados["taxa_tabagismo"] = agregados["casos_tabagistas"] / agregados["casos_srag"]
555 agregados["taxa_zona_rural"] = agregados["casos_zona_rural"] / agregados["casos_srag"]
556 agregados["taxa_vacinacao_covid"] = agregados["casos_vacinados_covid"] / agregados["casos_srag"]
557 agregados["taxa_febre"] = agregados["casos_febre"] / agregados["casos_srag"]
558 agregados["taxa_tosse"] = agregados["casos_tosse"] / agregados["casos_srag"]
559 agregados["indice_fluxo_referencia_hospitalar"] = agregados["casos_referenciados"] / agregados["casos_srag"]
560
561 print("SRAG: OK")
562 return agregados
563
564 """## Etapa 3 – Leitos (filtrada para SP)
565
566 Sem mudança na granularidade temporal (leitos é um dado por município, não por período). Mantém
567 leitos por especialidade de UTI, proporção SUS e contagem de hospitais distintos.
568 """
569
570 def carregar_leitos():
571     print("Carregando Leitos...")
572     df = ler_csv_seguro(os.path.join(PASTA_DADOS, "Leitos_2025.csv"))
573
574     coluna_uf = "UF"
575     coluna_municipio = "MUNICIPIO"
576     coluna_leitos = "LEITOS_SUS"
577     coluna_leitos_existentes = "LEITOS_EXISTENTES"
578     coluna_cnes = "CNES"
579
580     for nome_var, coluna in [
581         ("coluna_uf", coluna_uf), ("coluna_municipio", coluna_municipio), ("coluna_leitos", coluna_leitos),
582     ]:
583         if coluna not in df.columns:
584             print(f"\nA coluna '{coluna}' (usada como {nome_var}) não existe no arquivo.")
585             print("Colunas disponíveis:", list(df.columns))
586             raise KeyError(f"Ajuste {nome_var} em carregar_leitos() com um dos nomes acima.")
587
588     df = df[df[coluna_uf] == UF_ALVO]
589     df["municipio"] = df[coluna_municipio].apply(normalizar_municipio)
590
591     df[coluna_leitos] = pd.to_numeric(df[coluna_leitos], errors="coerce")
592
593     colunas_uti_especialidade = {
594         "UTI_TOTAL_SUS": "leitos_uti_sus",
595         "UTI_ADULTO_SUS": "leitos_uti_adulto_sus",
596         "UTI_PEDIATRICO_SUS": "leitos_uti_pediatico_sus",
597         "UTI_NEONATAL_SUS": "leitos_uti_neonatal_sus",
598     }
599     agregacoes = {coluna_leitos: "sum"}
600     for coluna_original in colunas_uti_especialidade:
601         if coluna_original in df.columns:
602             df[coluna_original] = pd.to_numeric(df[coluna_original], errors="coerce")
603             agregacoes[coluna_original] = "sum"
604         else:
605             print(f"Aviso: coluna '{coluna_original}' não encontrada – pulando essa especialidade.")
606
607     if coluna_leitos_existentes in df.columns:
608         df[coluna_leitos_existentes] = pd.to_numeric(df[coluna_leitos_existentes], errors="coerce")
609         agregacoes[coluna_leitos_existentes] = "sum"
610     else:
611         print(f"Aviso: coluna '{coluna_leitos_existentes}' não encontrada – não será possível calcular proporcao_leitos_sus.")
612
613     ocupacao = df.groupby("municipio").agg(agregacoes).reset_index()
614     ocupacao = ocupacao.rename(columns=colunas_uti_especialidade)
615
616     if coluna_leitos_existentes in ocupacao.columns:
617         ocupacao["proporcao_leitos_sus"] = ocupacao[coluna_leitos] / ocupacao[coluna_leitos_existentes]
618
619     if coluna_cnes in df.columns:
620         hospitais_sus = (
621             df[df[coluna_leitos] > 0]
622             .groupby("municipio")[coluna_cnes]
623             .nunique()
624             .reset_index(name="n_hospitais_sus")
625         )
626         ocupacao = ocupacao.merge(hospitais_sus, on="municipio", how="left")
627         ocupacao["n_hospitais_sus"] = ocupacao["n_hospitais_sus"].fillna(0)
628     else:
629         print(f"Aviso: coluna '{coluna_cnes}' não encontrada – não será possível calcular n_hospitais_sus.")
630
631     print("Leitos: OK")
632     return ocupacao
633
634 """## Etapa 4 – População (IBGE)
635
636 O JSON vem no formato aninhado da API do IBGE (SIDRA).
637 """
638
639 def carregar_populacao():
640     caminho = os.path.join(PASTA_DADOS, "populacao_municipios_2025 (1).json")
641     print("Carregando População...")
642     with open(caminho, encoding="utf-8") as f:
643         dados = json.load(f)
644
645     series = dados[0]["resultados"][0]["series"]
646
647     registros = []
648     for item in series:
649         nome_completo = item["localidade"]["nome"]
650         populacao = item["serie"].get("2025")
651         if " - " in nome_completo:
652             municipio, uf = nome_completo.rsplit(" - ", 1)
653         else:
654             municipio, uf = nome_completo, None

```

```

655     registros.append({
656         "municipio": normalizar_municipio(municipio),
657         "uf": uf,
658         "populacao": int(populacao) if populacao is not None else None,
659     })
660
661     populacao_df = pd.DataFrame(registros)
662     populacao_df = populacao_df[populacao_df["uf"] == UF_ALVO]
663     print("População: OK")
664     return populacao_df[["municipio", "populacao"]]
665
666 """## Classificação regional (Norte / Sul / Leste / Oeste / Centro)
667
668 Reaproveita a geocodificação já feita para a imputação de temperatura – cada município é
669 classificado com base na posição da sua coordenada em relação ao **centro geográfico do estado**
670 (mediana das coordenadas de todos os municípios do dataset), não em relação à capital.
671
672 **Como funciona:** calcula o centro (mediana de latitude/longitude) e uma faixa central em torno
673 dele. Município dentro dessa faixa nos dois eixos vira ``Centro``. Fora da faixa, compara se o
674 desvio é maior em latitude (Norte/Sul) ou em longitude (Leste/Oeste) e classifica pelo eixo
675 dominante. Município sem coordenada encontrada vira ``Não informado``.
676 """
677
678 def geocodificar_municipios(lista_municipios, pausa_segundos=1.0):
679     """Descobre a latitude/longitude de cada município via OpenStreetMap
680     (Nominatim). Respeita 1 requisição por segundo (limite do serviço gratuito)."""
681     from geopy.geocoders import Nominatim
682     geolocator = Nominatim(user_agent="vigiar_etl_fiap")
683     coordenadas = {}
684     for i, municipio in enumerate(lista_municipios, start=1):
685         try:
686             local = geolocator.geocode(f"{municipio}, São Paulo, Brasil", timeout=10)
687             coordenadas[municipio] = {"lat": local.latitude, "lon": local.longitude} if local else {"lat": None, "lon": None}
688         except Exception:
689             coordenadas[municipio] = {"lat": None, "lon": None}
690     time.sleep(pausa_segundos)
691     if i % 50 == 0:
692         print(f"    {i}/{len(lista_municipios)} municípios geocodificados")
693     return coordenadas
694
695
696 def classificar_regiao(municipios_coords):
697     """Classifica cada município em Norte/Sul/Leste/Oeste/Centro, com base na
698     posição em relação à mediana das coordenadas de todos os municípios (o
699     'centro geográfico' do conjunto de dados)."""
700     lats = [c["lat"] for c in municipios_coords.values() if c["lat"] is not None]
701     lons = [c["lon"] for c in municipios_coords.values() if c["lon"] is not None]
702     if not lats or not lons:
703         return {m: "Não informado" for m in municipios_coords}
704
705     lat_central, lon_central = np.median(lats), np.median(lons)
706     margem_lat = (max(lats) - min(lats)) / 6 or 0.01
707     margem_lon = (max(lons) - min(lons)) / 6 or 0.01
708
709     regioes = {}
710     for municipio, coord in municipios_coords.items():
711         if coord["lat"] is None or coord["lon"] is None:
712             regioes[municipio] = "Não informado"
713             continue
714         delta_lat = coord["lat"] - lat_central
715         delta_lon = coord["lon"] - lon_central
716         if abs(delta_lat) < margem_lat and abs(delta_lon) < margem_lon:
717             regioes[municipio] = "Centro"
718         elif abs(delta_lat) / margem_lat >= abs(delta_lon) / margem_lon:
719             regioes[municipio] = "Norte" if delta_lat > 0 else "Sul"
720         else:
721             regioes[municipio] = "Leste" if delta_lon > 0 else "Oeste"
722     return regioes
723
724 """## Grade completa de município x semana + união final
725
726 Antes de unir, criamos a **grade completa** de município x semana – todo município tem uma linha em
727 toda semana do período, mesmo sem notificação de SRAG (nesse caso, contagens viram 0, em vez da
728 linha simplesmente não existir). População, leitos e temperatura continuam podendo ficar vazios
729 quando não há dado real disponível.
730 """
731
732 def completar_grade_e_unir(srag, populacao, leitos, temperatura):
733     # Filtro adicionado após investigação: como "municipio" agora vem do município de
734     # RESIDÊNCIA (ID_MN_RESI), pacientes que moram em outro estado mas foram atendidos em
735     # SP (filtro de UF é sobre a notificação, não a residência) entravam na base. Isso não
736     # só inflava o dataset com municípios fora de escopo – contaminava a geocodificação e
737     # a classificação regional: max(lat)/min(lat) passava a medir a extensão do BRASIL
738     # inteiro em vez de só SP, e a margem de "Centro" ficava tão grande que 91% dos
739     # municípios paulistas caíam em "Centro" e Norte/Sul ficavam vazios. Filtrando aqui,
740     # antes de montar todos_municipios, o problema é resolvido na raiz – tanto pro dataset
741     # final quanto pra geocodificação/classificação regional que usa essa mesma lista.
742     municipios_sp = set(populacao["municipio"])
743     n_municipios_antes = srag["municipio"].nunique()
744     n_linhas_antes = len(srag)
745     srag = srag[srag["municipio"].isin(municipios_sp)].copy()
746     n_municipios_removidos = n_municipios_antes - srag["municipio"].nunique()
747     n_linhas_removidas = n_linhas_antes - len(srag)
748     if n_municipios_removidos > 0:
749         print(f"SRAG: removidos {n_municipios_removidos} municípios ({n_linhas_removidas} linhas) "
750               f"sem correspondência na tabela de população – residência fora de SP ou não informada.")
751
752     todos_municipios = pd.concat([
753         srag["municipio"], populacao["municipio"], leitos["municipio"]
754     ]).unique()
755     semanas = pd.period_range(srag["semana"].min(), srag["semana"].max(), freq="W")
756
757     grade = pd.MultiIndex.from_product(
758         [todos_municipios, semanas], names=["municipio", "semana"]
759     ).to_frame(index=False)
760
761     dataset = grade.merge(srag, on=["municipio", "semana"], how="left")
762     dataset = dataset.merge(populacao, on="municipio", how="left")
763     dataset = dataset.merge(leitos, on="municipio", how="left")
764     dataset = dataset.merge(temperatura, on=["municipio", "semana"], how="left")
765
766     colunas_contagem = [

```

```

767 "casos_srag", "casos_influenza", "casos_covid", "obitos",
768 "casos_uti", "casos_com_comorbidade", "casos_vacinados", "casos_idosos",
769 "taxa_obito", "taxa_uti", "taxa_comorbidade", "taxa_vacinados", "taxa_idosos",
770 # novas variáveis: contagens e taxas (0 é o valor correto quando não há
771 # casos na semana). As colunas de dias (tempo_medio_uti_dias,
772 # atraso_* dias, tempo_ate_desfecho_dias) ficam de fora de propósito -
773 # continuam NaN quando não há caso, porque "0 dias de atraso" seria
774 # uma informação errada, não "não houve caso".
775 "casos_surto", "casos_saturacao_baixa", "casos_dispnea", "casos_desc_resp",
776 "casos_vent_invasiva", "casos_pcr_positivo", "casos_antigenico_positivo",
777 "casos_sars2", "casos_vsr", "casos_testados", "casos_fator_risco",
778 "casos_gestantes", "casos_tabagistas", "casos_zona_rural",
779 "casos_vacinados_covid", "casos_febre", "casos_tosse", "casos_referenciados",
780 "taxa_cadeia_surto", "taxa_saturacao_baixa", "taxa_dispnea",
781 "taxa_desc_respiratorio", "taxa_ventilacao_invasiva",
782 "taxa_positividade_pcr", "taxa_positividade_antigenico", "taxa_sars2",
783 "taxa_vsr", "taxa_testagem", "taxa_fator_risco", "taxa_gestantes",
784 "taxa_tabagismo", "taxa_zona_rural", "taxa_vacinacao_covid",
785 "taxa_febre", "taxa_tosse", "indice_fluxo_referencia_hospitalar",
786 ]
787 for coluna in colunas_contagem:
788     if coluna in dataset.columns:
789         dataset[coluna] = dataset[coluna].fillna(0)
790
791 return dataset, todos_municipios
792
793 """## Geração do dataset final
794
795 Une tudo, geocodifica os municípios (uma única vez, reaproveitada para região e temperatura),
796 imputa temperatura ausente por proximidade, calcula colunas derivadas e formata as datas.
797 """
798
799 def preencher_temperatura_por_proximidade(dataset, temperatura, estacoes_coords, municipios_coords):
800     """Para linhas sem temp_media, usa a temperatura da estação mais próxima
801     do município (calculada por coordenada), marcando a linha em
802     'temp_imputada'."""
803     dataset = dataset.copy()
804     dataset["temp_imputada"] = dataset["temp_media"].isna()
805
806     def estacao_mais_proxima(lat, lon):
807         melhor, menor_dist = None, float("inf")
808         for nome_estacao, info in estacoes_coords.items():
809             if info["lat"] is None or info["lon"] is None:
810                 continue
811             R = 6371
812             dlat, dlon = radians(info["lat"] - lat), radians(info["lon"] - lon)
813             a = sin(dlat / 2) ** 2 + cos(radians(lat)) * cos(radians(info["lat"])) * sin(dlon / 2) ** 2
814             dist = R * 2 * atan2(sqrt(a), sqrt(1 - a))
815             if dist < menor_dist:
816                 menor_dist, melhor = dist, nome_estacao
817         return melhor
818
819     mapa_estacao_mais_proxima = {}
820     for municipio in dataset.loc[dataset["temp_imputada"], "municipio"].unique():
821         coord = municipios_coords.get(municipio, {"lat": None, "lon": None})
822         if coord["lat"] is None:
823             mapa_estacao_mais_proxima[municipio] = None
824         else:
825             mapa_estacao_mais_proxima[municipio] = estacao_mais_proxima(coord["lat"], coord["lon"])
826
827     def buscar_temp_proxima(linha):
828         if not linha["temp_imputada"]:
829             return linha["temp_media"]
830         estacao_proxima = mapa_estacao_mais_proxima.get(linha["municipio"])
831         if estacao_proxima is None:
832             return None
833         valores = temperatura.loc[
834             (temperatura["municipio"] == estacao_proxima) & (temperatura["semana"] == linha["semana"]),
835             "temp_media"
836         ]
837         return valores.iloc[0] if len(valores) else None
838
839     dataset["temp_media"] = dataset.apply(buscar_temp_proxima, axis=1)
840     n_imputadas = dataset.loc[dataset["temp_imputada"], "temp_media"].notna().sum()
841     print(f'{n_imputadas} de {dataset["temp_imputada"].sum()} linhas preenchidas por proximidade.')
842     return dataset
843
844
845 def gerar_dataset_final():
846     print("Carregando e tratando cada fonte (recorte: São Paulo)...")
847     temperatura, estacoes_coords = carregar_temperatura_sp()
848     srag = carregar_srag()
849     leitos = carregar_leitos()
850     populacao = carregar_populacao()
851
852     print("Completando a grade de município x semana (semanas sem SRAG viram 0)...")
853     dataset, todos_municipios = completar_grade_e_unir(srag, populacao, leitos, temperatura)
854
855     print(f'Geocodificando {len(todos_municipios)} municípios (usado para região + temperatura)...')
856     municipios_coords = geocodificar_municipios(todos_municipios)
857
858     print("Classificando região de cada município...")
859     mapa_regiao = classificar_regiao(municipios_coords)
860     dataset["regiao"] = dataset["municipio"].map(mapa_regiao).fillna("Não informado")
861
862     print("Preenchendo temperaturas ausentes por proximidade geográfica...")
863     dataset = preencher_temperatura_por_proximidade(dataset, temperatura, estacoes_coords, municipios_coords)
864
865     print("Calculando colunas derivadas (sazonalidade, capacidade, tendência)...")
866     dataset = dataset.sort_values(["municipio", "semana"]).reset_index(drop=True)
867
868     dataset["casos_por_10k"] = dataset["casos_srag"] / dataset["populacao"] * 10_000
869     dataset["leitos_por_1000_hab"] = dataset["LEITOS_SUS"] / dataset["populacao"] * 1000
870
871     dataset["data_inicio_semana"] = dataset["semana"].dt.start_time
872     dataset["ano"] = dataset["data_inicio_semana"].dt.year
873     dataset["semana_numero"] = dataset["data_inicio_semana"].dt.isocalendar().week
874     dataset["data_formatada"] = dataset["data_inicio_semana"].apply(formatar_data_br)
875
876     dataset["casos_srag_semana_anterior"] = dataset.groupby("municipio")["casos_srag"].shift(1)
877     dataset["media_movel_3_semanas_casos"] = dataset.groupby("municipio")["casos_srag"].transform(
878         lambda serie: serie.shift(1).rolling(window=3, min_periods=1).mean()

```

```

879     )
880     dataset["variacao_temp"] = dataset.groupby("municipio")["temp_media"].diff()
881     dataset["variacao_casos_pct"] = (
882         dataset.groupby("municipio")["casos_srag"].pct_change()
883         .replace([float("inf"), float("-inf")], None)
884     )
885
886     dataset = dataset.drop(columns=["semana", "data_inicio_semana"])
887
888     colunas_finais = [
889         "municipio", "regiao", "regional_saude", "ano", "semana_numero", "data_formatada",
890         "casos_srag", "casos_influenza", "casos_covid", "obitos", "casos_uti",
891         "casos_com_comorbidade", "casos_vacinados", "casos_idosos",
892         "taxa_obito", "taxa_uti", "taxa_comorbidade", "taxa_vacinados", "taxa_idosos",
893         "populacao", "LEITOS_SUS", "leitos_uti_sus", "leitos_uti_adulto_sus",
894         "leitos_uti_pediatico_sus", "leitos_uti_neonatal_sus", "LEITOS_EXISTENTES",
895         "proporcao_leitos_sus", "n_hospitais_sus", "leitos_por_1000_hab",
896         "temp_media", "temp_imputada", "casos_por_10k",
897         "casos_srag_semana_anterior", "media_movel_3_semanas_casos",
898         "variacao_temp", "variacao_casos_pct",
899         # 24 novas variáveis (tabela de features anexada na conversa)
900         "taxa_cadeia_surto", "taxa_saturacao_baixa", "taxa_dispneia",
901         "taxa_desc_respiratorio", "taxa_ventilacao_invasiva", "tempo_medio_uti_dias",
902         "taxa_positividade_pcr", "taxa_positividade_antigenico", "taxa_sars2", "taxa_vsr",
903         "taxa_testagem", "atraso_notificacao_dias", "atraso_coleta_dias",
904         "indice_fluxo_referencia_hospitalar", "atraso_internacao_dias",
905         "taxa_fator_risco", "taxa_gestantes", "taxa_tabagismo", "taxa_zona_rural",
906         "taxa_vacinacao_covid", "taxa_febre", "taxa_tosse",
907         "atraso_digitacao_dias", "tempo_ate_desfecho_dias",
908     ]
909     colunas_finais = [c for c in colunas_finais if c in dataset.columns]
910     dataset = dataset[colunas_finais]
911
912     dataset.to_csv(SAIDA, index=False)
913     print(f"\nDataset final salvo em: {SAIDA}")
914     print(f"Municípios únicos: {dataset['municipio'].nunique()}")
915     print(f"Total de linhas: {len(dataset)}")
916     return dataset
917
918
919     dataset_final = gerar_dataset_final()
920     display(dataset_final.head())
921
922     """## Download do resultado"""
923
924     from google.colab import files
925
926     files.download(SAIDA)

```